УНИВЕРЗИТЕТ У БАЊОЈ ЛУЦИ

ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ

Оливер Ерић

РАЗВОЈ МИКРОСЕРВИСА У ПРОГРАМСКОМ ЈЕЗИКУ RUST

дипломски рад

Бања Лука, април 2026.

Тема: **РАЗВОЈ МИКРОСЕРВИСА У ПРОГРАМСКОМ ЈЕЗИКУ RUST**

Кључне ријечи: **информациони системи, сервисно-оријентисана архитектура, микросервиси, Rust**

Комисија: **проф. др Милош Љубојевић, предсједник**

проф. др Михајло Савић, ментор

Игор Дујловић, ма, члан

Кандидат:

Оливер Ерић

УНИВЕРЗИТЕТ У БАЊОЈ ЛУЦИ
ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ
КАТЕДРА ЗА РАЧУНАРСТВО И ИНФОРМАТИКУ

Предмет: Пројектовање информационих система у интернет окружењу

Тема: РАЗВОЈ МИКРОСЕРВИСА У ПРОГРАМСКОМ ЈЕЗИКУ RUST

Задатак: Дати преглед и описати еволуцију архитектура информационих система с посебним освртом на сервисно-оријентисану и микросервисну архитектуру. Описати специфичности и ограничења микросервисне архитектуре. Дати кратак преглед програмског језика Rust, као и развојних оквира и библиотека погодних за развој микросервиса. Одабрати и детаљно описати једну од представљених технологија. У практичном дијелу рада реализовати систем који илуструје употребу претходно описаних технологија.

Ментор: проф. др Михајло Савић

Кандидат: Оливер Ерић (1175/17)

Лиценца: □ CC-BY ■ CC-BY-SA □ CC-BY-NC □ CC-BY-NC-SA

Бања Лука, април 2026.

САДРЖАЈ

1. Увод ... 1
2. Архитектуре информационих система ... 3
   2.1. Еволуција архитектура информационих система ... 3
   2.2. Сервисно-оријентисана архитектура ... 5
      2.2.1. Сервисно-оријентисани принципи дизајна ... 5
      2.2.2. Циљно стање сервисно-оријентисаног приступа ... 6
      2.2.3. Инфраструктурна проширења у SOA приступу ... 7
      2.2.4. Веб сервиси засновани на WSDL документима и SOAP порукама ... 9
      2.2.5. REST и *RESTful* сервиси ... 10
   2.3. Микросервисна архитектура ... 11
      2.3.1. Моделовање микросервиса ... 13
      2.3.2. Дизајн приступне тачке микросервисном систему ... 14
      2.3.3. Комуникација микросервиса ... 14
      2.3.4. Микросервисно управљање подацима ... 19
      2.3.5. Отпорност и висока доступност микросервиса ... 20
      2.3.6. Оркестратори и контејнеризација микросервиса ... 20
      2.3.7. Service Mesh ... 22
      2.3.8. DevOps ... 22
      2.3.9. Предности и недостаци микросервисног приступа ... 24
   2.4. Микросервисни у поређењу са SOA приступом ... 26
3. Преглед програмског језика Rust ... 28
   3.1. Ownership систем ... 28
   3.2. Типови ... 31
      3.2.1. Непостојање *null* вриједности ... 32
   3.3. Енкапсулација и систем модула ... 33
   3.4. Управљање грешкама ... 33
   3.5. Генерици ... 34
   3.6. Traits и trait bound ... 34
   3.7. Конкурентност ... 35
      3.7.1. Async систем ... 35
   3.8. Предности и недостаци програмског језика Rust ... 36
4. Развој микросервиса у програмском језику Rust ... 37
   4.1. Сервер и клијент HTTP-а ... 37
   4.2. Серијализација и десеријализација података ... 39
   4.3. Клијент релационе базе података и ORM ... 39
   4.4. Клијент система за размјену порука ... 40

4.5. Аутентификација, ауторизација и остали сигурносни аспекти 40
4.6. Опсервабилност 40
4.7. Конфигурабилност 41
5. Практични рад 42
5.1. Опис апликације 42
5.2. Спецификација и архитектура апликације 42
5.3. Процес развоја 45
5.3.1. Веб развојни оквир 45
5.3.2. Клијент HTTP протокола 49
5.3.3. Серијализација и десеријализација података 50
5.3.4. SQL клијент 51
5.3.5. Клијент *RabbitMQ* брокера 54
5.3.6. Аутентификација и остали сигурносни аспекти 57
5.3.7. Опсервабилност 59
5.3.8. Конфигурабилност 61
5.4. Развијено рјешење 62
6. Закључак 64
Литература 65

ОЗНАКЕ И СКРАЋЕНИЦЕ

Ознака/скраћеница	Значење
(R)IS	(Рачунарски) Информациони систем
UI	User Interface
SOA	Service-Oriented Architecture
API	Application Programming Interface
IT	Information Technology
ESB	Enterprise Service Bus
REST	Representational State Transfer
gRPC	Google Remote Procedure Call
GraphQL	Graph Query Language
WSDL	Web Service Description Language
SOAP	Simple Object Access Protocol
URI	Unified Resource Locator
XML	Extensible Markup Language
WS	Web Service
TLS	Transport Layer Security
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
CI/CD	Continuous Integration/Continuous Delivery
DDD	Domain-Driven Design
AMQP	Advanced Message Queueing Protocol
OSI	Open Systems Interconnection
CAP	Consistency, Atomicity, Partition Tolerance
QA	Quality Assurance
SPA	Single Page Application
OS	Operating System
SSE	Server Sent Events
ORM	Object-Relational Mapping

1. Увод

Почевши од класичних монолитних система који обављају све функционалности унутар једног процеса, преко слојевитих система са хијерархијским поретком до потпуно декомпонованих система на скуп сервиса, информациони системи постају све слабије спрегнути, а све потпомогнуто моћнијом мрежом и с циљем постизања веће поузданости, дистрибуираности и скалабилности. Сервисно-оријентисана архитектура је начин дизајнирања софтвера који подразумијева декомпоновање система на независне сервисе који остварују функционалности система комуницирајући међусобно путем мрежних сервиса. Један специфичан приступ сервисно-оријентисаној архитектури је микросервисна архитектура, која подразумијева дефинисање независних сервиса ограничених могућности на основу пословних функција. Циљ је постићи флексибилност по питању развоја, испоруке, скалирања и избора конкретне технологије за имплементацију. Једна од технологија је програмски језик Rust, који је дизајниран имавши у виду захтјеве конкурентности те са својом *memory-safe* карактеристиком омогућава писање ефикасних, сигурних и поузданих апликација. Циљ овог рада је описати како се програмски језик Rust може искористити, употребом којих библиотека и развојних оквира, за развој скалабилне микросервисне апликације.

Друга глава је посвећена информационим системима, њиховим архитектурама, еволуцији архитектура те предностима и манама сваке од њих. Затим су детаљно описане сервисно-оријентисана и микросервисна архитектура. Везано за сервисно-оријентисану архитектуру, наведени су њени принципи дизајна, предности које пружа пословним окружењима те су уопштено описане неке њене имплементације као што су SOAP веб и *RESTful* сервиси. Затим су дате специфичности и ограничења микросервисне архитектуре. Наведена је једна метода из доменом-вођеног дизајна која се може искористити за њихово моделовање. Описани су разни начини комуникације микросервиса употребом различитих протокола. Објашњен је специфичан приступ управљања подацима, проблеме који он ствара и начин на који се они могу ријешити. Додатно, наведене су и одређене погодности које се могу користити приликом развоја и експлоатације сервиса као што су контејнеризација, *Service Mesh* и *DevOps*. На крају друге главе дато је поређење SOA и микросервисног приступа и њихови специфични случајеви употребе.

У трећој глави је дат кратак увод у програмски језик Rust. Описани су главни концепти као што су *zero-cost* апстракције и начин да се осигура сигурност меморије употребом специфичног *ownership* система. Објашњено је зашто Rust не посједује *null* вриједност и како он користи строги систем типова да представи ситуацију непостојања вриједности. Такође, дат је начин на који се управља грешкама у програму употребом система типова. Наведени су начини постизања конкурентности употребом нити и *async-await* механизма. На крају треће главе дате су предности и недостаци Rust програмског језика.

У четвртој глави дат је кратак опис Rust библиотека и развојних оквира који се могу искористити за развој микросервиса. Ово укључује веб развојне оквире те библиотеке за серијализацију и десеријализацију података, аутентификацију, ауторизацију, клијенте и сервере разних протокола (SQL, AMQP, HTTP) те конфигурабилност и опсервабилност апликације.

Пета глава је посвећена практичном раду. На почетку је дат опис реализоване апликације те њена груба спецификација и архитектура. Наведени су сви конкретни кориштени микросервиси и њихове функционалности. Затим је детаљно описан процес развоја од кориштених техника и алата до кориштених функционалности конкретних библиотека и развојних оквира. Функционалности укључују читав процес прихватања, парсирања, рутирања и обраде захтјева. Описано је како десеријализовати и серијализовати податке те како користити клијенте према екстерним системима као што су SQL база података и *RabbitMQ* брокер порука. Покривени си у сигурносни аспекти као што су аутентификација употребом JWT-а и хеширање лозинки. Такође, описан је начин постизања потпуно опсервабилне дистрибуиране апликације употребом *Jaeger* система и *OpenTelemetry* стандарда. На крају пете главе је описано како се апликација може конфигурисати.

У закључку су наведене предности и недостаци микросервисне у односу на монолитну архитектуру. Затим су наведени потребни услови за развој микросервисне апликације. Објашњено је зашто Rust програмски језик испуњава ове услове. На крају је дат примјер апликације која је предмет практичног рада те је објашњено како је Rust са свим својим библиотекама и развојним оквирима искориштен за развој ове апликације.

2. Архитектуре информационих система

Информациони систем (ИС) је формални, друштвено-технолошки и организациони систем намијењен за прикупљање, обраду, похрану и дистрибуцију информација. Најуопштеније посматрано, састоји се из људи, задатака, улога и технолошке компоненте. Једна његова специјализација је рачунарски информациони систем (РИС) чија технолошка компонента укључује рачунарске ресурсе [1].

Архитектура система је подјела система на његове главне дијелове са највишег нивоа апстракције те њихов начин интеракције [2]. Архитектура РИС-а се односи на архитектуру његове рачунарске компоненте која има директан или индиректан утицај на све остале компоненте. Стога је адекватан одабир врсте архитектуре од суштинског значаја за цјелокупан РИС. Најчешће кориштене врсте архитектура РИС-а су:

- Монолитна
- Клијент-Сервер (CS)
- Трослојна
- *Peer-to-peer* (P2P)
- Сервисно-оријентисана (SOA)
- Микросервисна

2.1. Еволуција архитектура информационих система

Класична монолитна архитектура је софтверска архитектура гдје су све функционалности које систем пружа имплементиране и које се извршавају као једна софтверска јединица. Ова јединица је најчешће један ОС процес. Компоненте су јако спрегнуте тако да су овакви системи тешки за одржавање, скалирање и независни *deployment*. Да би се постигле ове особине дошло је до раслојавања архитектуре.

Техника раслојавања архитектуре ИС-а користи инжењерски принцип који подразумијева да се комплексан проблем који се рјешава реализацијом система, раздвоји на мање, једноставније цјелине те свака од тих цјелина ријеши засебно [1]. Слојевита архитектура је архитектура која дијели систем на скуп слојева одређених функционалности који су хијерархијски организовани, што подразумијева да виши слојеви користе сервисе нижих слојева док нижи слојеви немају информацију о постојању виших слојева [2]. Први слојевити системи који су настали су двослојни системи базирани на клијент-сервер моделу у којима је, уобичајено, клијентски слој апликације вршио апликациону логику и радње везане за интеракцију са корисником, док је серверски слој био база података [1]. Подјела одговорности на више слојева омогућава реализацију сваке од компонената ИС-а на више одвојених хардверских

чворова. Ово уједно представља потребан услов за хоризонтално скалирање система које омогућава кориштење система од стране великог броја корисника истовремено путем мреже. Додатно, ова архитектура постиже портабилност корисничког интерфејса на више платформи и независно еволуирање обје компоненте [6]. За разлику од монолитних, двослојни системи имају већу скалабилност јер није потребно скалирати цијели систем за повећање перформанси одређене функционалности већ само серверски слој.

Овакви системи су добро функционисали ако су захтјеви система били једноставно читање и ажурирање података из базе података. Међутим, сва пословна логика се уграђивала или у клијентски слој унутар UI кода или у серверски слој у виду похрањених процедура што није ишло у прилог одрживости кода [2]. Серверски дио оваквих система је најчешће био релациона база података док је све чешће у употреби била објектно-оријентисана парадигма која је била погодна за примјену пословних правила, валидацију улаза, вршење разних калкулација итд. [2]. Све ово доводи до додатног раслојавања клијентског слоја на презентациони и доменски слој. Доменски слој извршава сву комплексну доменску логику, врши превођење између објектно-оријентисаног и релационог начина презентовања података и њихових веза те је погодна локација за извршавање других радњи као што су контрола приступа и надзор сигурности [1]. Растом популарности веба овај модел постаје све чешће у примјени јер је одговарао подјели *browser*-сервер-база те омогућавао потпуну замјену презентационог слоја без измјене доменског слоја [2].

У модерном контексту, монолитна архитектура подразумијева да су све функционалности доменског слоја груписане у једну софтверску јединицу док је и даље цјелокупан систем трослојне архитектуре.

Поред ових архитектура, паралелно у примјени је и *peer-to-peer* (P2P) архитектура у којој су, за разлику од CS архитектуре, сви учесници система равноправни у смислу да су сви и пружатељи сервиса и конзументи сервиса других учесника [1]. Крајња функционалност система се остварује међусобном комуникацијом више учесника гдје сваки учесник може да самостално пружа неопходне ресурсе тако да није потребно да постоји улога централног ауторитета што уклања јединствену тачку отказа.

Уколико би пословна окружења аутоматизовала пословне задатке на серверској страни примјеном ових архитектура, утолико би долазила у ситуацију да приликом појаве нових захтјева или пословних процеса морају поново имплементирати старе функционалности или креирати потпуно нове апликације. Ако би се овај начин прилагођавања учестало користио долазило би до редунданције функционалности, неефикасности посла, тежег одржавања и администрације система те као резултат, већег IT трошка цјелокупног пословања. Као одговор на ове проблеме појављује се сервисно-оријентисана архитектура која заговара поновну употребљивост и постиже већи ниво скалабилности и поузданости система [3].

2.2. Сервисно-оријентисана архитектура

Оријентисаност ка сервисима је парадигма дизајна логике дистрибуираних рјешења која примјењујући принцип подјеле одговорности групише логику у независне логичке јединице које се називају сервиси [3]. Сваки сервис логички представља пословну могућност у пословном процесу и има јасно дефинисан излаз. Састоји се од једне или више пословних функција које заједно доприносе његовој крајњој сврси. Овај услов спречава партиционисање логике рјешења на велики број сервиса при чему би сваки сервис могао представљати функционалност, колико год она минорна била [1].

2.2.1. Сервисно-оријентисани принципи дизајна

Сервисно-оријентисана парадигма се може примијенити над софтверским програмима чинећи их на тај начин сервисима употребом сљедећих принципа дизајна [3]:

- Стандардизован сервисни уговор – Сервис кроз сервисни уговор на конзистентан начин оглашава своју сврху и могућности које пружа.
- Сервиси слабе спреге – Нагласак је на смањивању зависности између сервисног уговора, његове имплементације и конзумената сервиса. Слаба спрега омогућава постизање тога да пројектовање, реализација и еволуција сваког од сервиса буду независни од остатка система [1].
- Апстракција сервиса – Сервисни уговор треба да учини доступним само оне информације које су неопходне за конзумирање сервиса. Тежња је сакрити што је више могуће информација о унутрашњим процесима сервиса. Сакривањем имплементационих детаља уједно се олакшава постизање слабије спреге.
- Поновна употребљивост сервиса – Сервис треба да буде дизајниран тако да он уопште нема или има веома ограничено знање о генералнијем проблему који се рјешава његовом употребом. Овакав дизајн га чини поновно употребљивим од стране различитих пословних процеса.
- Аутономија сервиса – Сервис треба да је самодовољан што значи да има смисла као једна реализована цјелина и функционално не зависи од других сервиса.
- Сервиси без стања – Сервиси чувају информације о стању само онда када је то пријеко потребно што олакшава постизање веће доступности и скалабилности.
- Откривеност сервиса – Сваки сервис садржи неопходне метаподатке помоћу којих га је могуће открити и исправно интерпретирати.

- Сервиси као композиционе јединице – Сервиси треба да буду погодни за учествовање у сложеним композиционим конфигурацијама и прије него што постану дио исте. Ово за посљедицу има да сервис може да садржи друге сервисе што за крајњег конзумента не мора бити ни видљиво [1].

Најуопштеније посматрано, софтверски програм је сервис ако испољава своје функционалности кроз API који је дио уговора сервиса [1]. Основни захтјеви које треба да испуни су доступност, поузданост и способност комуникације заједничким језиком [3]. Када се сервисно-оријентисани приступ примијени на архитектуру информационог система, сервиси су слабо спрегнуте апликационе компоненте које пружају своје услуге другим апликационим компонентама употребом заједничког језика у виду усаглашеног мрежног комуникационог протокола [1].

2.2.2. Циљно стање сервисно-оријентисаног приступа

Информациони систем који користи сервисно-оријентисани приступ за аутоматизацију пословних процеса то чини тако што примјењује сервисно-оријентисане принципе дизајна над софтверским компонентама који га чине. Овим се тежи постићи циљно стање система које чине сљедећи стратешки циљеви и бенефити [3]:

- Повећана међусобна интероперабилност компонената – Што су сервиси интероперабилнији то лакше размјењују информације. Сервисе који лако размјењују информације могуће је лакше и непрестано компоновати у разноврсне конфигурације за аутоматизацију великог распона пословних процеса, независно од тога ком пројекту припадају и у ком временском периоду су пројектовани. Циљу постизања високе међусобне интероперабилности сервиса доприноси, у већој или мањој мјери, примјена свих сервисно-оријентисаних принципа дизајна.
- Повећана федерација – Федеративно окружење је оно унутар којег су ресурси и апликације уједињени с тим да сви одржавају своју аутономију и самоуправљање. Унутар информационог система федерација се постиже тако што се користе стандардизовани сервиси подложни композицијама, који енкапсулирају један сегмент пословања те га путем стандардизованог уговора на конзистентан начин испољавају.
- Повећан број опција за избор провајдера технологије – Независност од конкретног провајдера технологије за имплементацију рјешења се постиже стандардизованим и интероперабилним компонентама што омогућава организацијама да прилагођавају, проширују или мијењају технологије без нарушавања цјелокупне техничке архитектуре.
- Повећана усклађеност између пословног и технолошког домена – Прецизност којом је пословна логика аутоматизована логиком рјешења често одређује успјешност испуњавања пословних захтјева. Сервиси прецизно енкапсулирају и репрезентују пословне моделе чија промјена

нужно не изискује промјену цијелог система већ само одређеног сервиса. Као резултат, техничка архитектура константно еволуира заједно са пословањем.

- Повећан поврат улагања (*Return of Investment*) – Услијед високе интероперабилности сервиси су вишенамјенски те као такви омогућавају организацијама већи профит.
- Повећана агилност организација – Сервиси су поновно употребљиви ресурси које је могуће неограничен број пута користити за аутоматизацију нових или измијењених пословних процеса. Ова чињеница омогућава краће вријеме потребно за пуштање производа на тржиште те се тако повећава ефикасност са којом организације могу одговорити на промјене у пословном домену.
- Смањено оптерећење од стране IT сектора – Примјеном сервисно-оријентисаних принципа дизајна смањује се редунданција, величина и цијена оперативности IT сектора унутар организације.

Постизањем ових циљева ствара се ефикаснији и агилнији IT сектор, који је мање бреме организацији а већи јој сарадник у постизању крајњих циљева пословања [3].

Сервисно-оријентисана архитектура (SOA) је пословно-вођена архитектура која садржи слабо спрегнуте сервисе као основне компоненте серверског дијела ИС-а. Слаба спрега компоненти омогућава да архитектура буде једноставнија и да је њене елементе могуће развијати независно, све док оне не нарушавају интерфејс путем којег комуницирају. Сервиси могу да користе дијељене компоненте али само ако та зависност није логичке природе (нпр. могу дијелити исту базу података).

Употреба SOA приступа омогућава креирање апликација повезивањем већег броја различитих и међусобно независних сервиса, који могу бити дијелови више различитих апликација, писани у различитим програмским језицима, употребом различитих приступа и од стране различитих организација. Све ово је могуће постићи захваљујући постојању јасне дефиниције сервиса која омогућава идентичну употребу сервиса од стране различитих конзумената. Да би конзументи били у стању да употребљавају овакве сервисе потребно је да се прилагоде дефиницији сервиса на начин да се усагласе око кориштеног скупа протокола за опис, размјену и парсирање порука употребом метаподатака [1].

Због тога што се SOA састоји од више сервиса, често је потребно да постоје додатне инфраструктурне компоненте да би се постигла већа поузданост, перформансе и скалабилност система [3].

2.2.3. Инфраструктурна проширења у SOA приступу

SOA приступ подразумијева да се пословни процес често аутоматизује компоновањем више сервиса у сервисну композицију па је стога, потребно да постоји

одређен вид контроле тока извршавања ове композиције да би се пословни процес исправно извршио [3]. Ову контролу, са одређеним нивоом централизованости, могу да извршавају сљедећи ентитети:

- Оркестратор – Специјализована компонента која има знање о пословним процесима које је потребно извршити што јој омогућава да спроводи контролу тока над њима тако што координише сервисе предефинисаним редослиједом те враћа одговор крајњем клијенту. Ова компонента потенцијално контролише вишеструке сценарије током извршавања који доводе до различитих композиционих конфигурација (нпр. услијед изузетака) [3]. Представља централни чвор система и тачку повезивања за сваку појединачну компоненту стварајући тако одређен ниво централизованости управљања системом.
- Скуп аутономних сервиса – Користећи децентрализовани модел контроле тока, сваки сервис у композицији има дјелимичну контролу извршавања пословног процеса. Крајња функционалност је посљедица међусобне сарадње сервиса. Овај приступ користи кореографију која представља вид композиције појединачних сервиса из глобалне и дистрибуиране перспективе [1].

С обзиром на то да су SOA системи дистрибуирани, често је потребно да окружење за извршавање композиција сервиса управља са сљедећим аспектима система [3]:

- Сигурност – Сервис је један процес на мрежи и као такав, изложен је малициозним корисницима. Због тога приступ сервису треба да је дозвољен само одређеним сервисима и крајњим клијентима. Контролу приступа могуће је постићи употребом аутентификације и ауторизације. Тајност и интегритет података те аутентификација сервиса у комуникацији између два сервиса се може остварити употребом стандардног протокола као што је TLS.
- Дистрибуиране трансакције – Одређене операције могу укључивати промјене стања више дистрибуираних сервиса које треба да буду извршене у цјелости или уопште неизвршене. Стога је потребна координација оваквих операција да би се одржала конзистентност података између сервиса.
- Поуздана комуникација – Због тога што су сервиси на мрежи склони отказима, потребно је обезбједити механизам који омогућава комуникацију између сервиса независно од тренутне доступности одредишног сервиса. Осим тога, често је потребно обезбједити гаранцију да су поруке испоручене дефинисан број пута и редослиједом којим су послате како би се спријечио губитак података и њихова неконзистентност између сервиса.

- Рутирање захтјева – Због тога што често постоји више инстанци неког сервиса, да би се постигла већа скалабилност и доступност, потребно је вршити рутирање ка адекватној инстанци сервиса на основу верзије, правила, оптерећења или контекста.

Ове функционалности могу да се реализују на различите начине, од којих је у традиционалним пословним системима најчешћа употреба јединствене централне компоненте која интегрише све ове функционалности. Један од начина на који се ова централна компонента може реализовати јесте употребом ESB шаблона, чија компонента се може, осим у сврхе координације сервиса, искористи и у ситуацијама у којима постоје *legacy* компоненте које је потребно интегрисати у систем без измјена њих самих. Ове ставке ESB постиже прилагођавањем или превођењем модела података, веза, те захтјева и одговора [1].

SOA архитектура се може имплементирати употребом различитих технологија и стандарда од којих су најпопуларнији [1]:

- Веб сервиси засновани на WSDL документима и SOAP порукама
- *RESTful* сервиси
- Системи засновани на размјени порука

2.2.4. Веб сервиси засновани на WSDL документима и SOAP порукама

Сервис је веб сервис ако га је могуће објавити, лоцирати и позвати путем веба. Идентификован је уз помоћ URI-ја и интерагује са другим сервисима преко протокола базираних на интернету. Веб сервиси имају велики ниво поновне употребљивости јер се њихова интероперабилност постиже кориштењем стандардних веб технологија које су широко подржане. Осим тога, они су веома популарни јер могу да апстрахују већ постојеће, застарјеле, нестандардне или *proprietary* технологије употребом стандардних веб технологија и пруже их на широку употребу [4].

Један од протокола за имплементацију веб сервиса је SOAP (*Simple Object Access Protocol*) који служи за размјену структурисаних порука у дистрибуираном окружењу. Поруке су у стандардном XML формату а транспорт поруке се најчешће врши HTTP-ом. Било која апликација на интернету која користи овај протокол, писана у било којем програмском језику и која се извршава на било којој платформи и оперативном систему може да постане веб сервис претварајући тако интернет у платформу за развој апликација [5]. За дефинисање, описивање и откривање SOAP веб сервиса користи се сервисни уговор у виду WSDL документа који дефинише како да клијент компонује захтјев, моделе података порука, политике приступа итд.

SOA имплементација која остварује мању спрегу између конзумента и сервисног уговора тако што не користи додатни међуслој као SOAP веб сервиси већ се директно ослања на HTTP семантику јесу *RESTful* сервиси.

2.2.5. REST и *RESTful* сервиси

RESTful сервиси су дистрибуирани системи који користе REST архитектурални стил који подразумијева да се систем дизајнира на начин да се на елементе архитектуре примјене одређена ограничења која се примарно односе на комуникацију у систему. У великој мјери користе SOA принципе и имају сличан циљ постизања пожељних својстава дистрибуираног система с тим да су стриктнија и формалнија. Ова архитектурална ограничења су [6]:

- Клијент-сервер архитектура – Клијент је тај који иницира комуникацију. Сервер чека на његове захтјеве које може обрадити или одбити прије него што му пошаље одговор [1].
- Непостојање стања – Сва комуникација се искључиво изводи без чувања стања клијента на серверу. Сам захтјев садржи све податке неопходне за његову обраду или надзор. Овим ограничењем систем постиже:
    1. Бољу видљивост – Сваки захтјев је самоописујући тако да систем за надзор има све потребне информације да утврди природу захтјева без прикупљања информација из других захтјева.
    2. Већу поузданост – Сваки захтјев је самосадржавајући па је стога лакше извршити опоравак од парцијалних отказа јер не постоје подаци о стању на серверској страни који се могу изгубити.
    3. Већу скалабилност - Због тога што клијент чува своје стање, сервер не мора да синхронизује то стање између више својих инстанци, тако да је тривијално извршити хоризонтално скалирање.

  Негативна страна је та што клијент понекад треба да пошаље више захтјева за извршавање једне операције.
- Кеширање – Приликом слања одговора сервер може да дефинише стратегију кеширања одређених података на страни клијента. Ову стратегију клијент може да искористи приликом вишеструког слања тако означених података. Овиме се повећавају перформансе јер сервер не мора увијек слати а клијент захтијевати репетитивне податке. Негативна страна је та што неажурност кешираних података на клијентској страни може довести до непоузданих резултата.
- Слојевита организација – Подразумијева да између клијента и сервера могу да се нађу посредници који транспарентно пресрећу саобраћај као што су *proxy*, *load balancer*, *firewall*, комплексне *middleware* компоненте итд. Ово ограничење омогућава слободно проширење система додавањем нових компонената те независан развој сваке од њих. Негативна страна је смањен одзив система јер захтјев у слојевитом систему може да буде процесиран од стране више компоненти.

- Униформност интерфејса – С циљем да постигне високу интероперабилност и ниску спрегу компонената REST дефинише сљедећа ограничења интерфејса [6]:

    1. Идентификација ресурса – Сви подаци којима клијент може да приступи или манипулише представљени су ресурсима а операције над њима се врше путем њихових јединствених идентификатора (нпр. URI-ја).

    2. Манипулација ресурсима путем репрезентације – Измјену стања ресурса или његово брисање клијент врши слањем поруке са жељеном репрезентацијом тог ресурса. Операције над ресурсима се извршавају на основу URI-ја и методе HTTP протокола.

    3. Самоописујуће поруке – Свака порука је јасна сама по себи и садржи све што је потребно за њено процесирање, без потребе за памћењем претходне интеракције компонената.

    4. HATEOAS (*Hypermedia As The Engine Of Application State*) – Сервер у одговору у виду хипермедије шаље све релевантне информације које су клијенту довољне да динамички навигира кроз сва стања апликације. Стога клијент нема потребу да памти више URI-ја приликом вршења операција већ само једног и то оног који је почетна приступна тачка систему. Ово додатно олакшава независно еволуирање њега и сервера [7].

RESTful сервиси могу да раде са различитим форматима поруке као што су JSON, XML, *plain text*, HTML. У односу на SOAP веб сервисе, имају мањи *overhead* док SOAP веб сервиси омогућавају већи ниво контроле постојањем додатног слоја поруке и употребом додатних стандарда. *RESTful* сервиси су више оријентисани ка флексибилности а мање ка стандардизацији.

REST стил се најчешће користи као један од начина имплементације комуникације у микросервисној архитектури.

2.3. Микросервисна архитектура

Подразумијева да елементе система чини скуп независних сервиса ограничених могућности који се називају микросервиси. Ова архитектура је примарно оријентисана ка *backend*-у, иако се може користити и за развој *frontend* дијела апликације [10].

За разлику од SOA приступа гдје сервис представља потпуну пословну могућност, микросервис је специјализован за извршавање једног пословног задатка. Дизајнира се тако да има висок ниво аутономије да би могао да буде развијан, скалиран, испоручен, одржаван и верзионисан независно од других микросервиса [10]. Штавише, заједно са интеграцијом, испорука се аутоматизује кроз процесе континуалне интеграције и испоруке (CI/CD). Као дио ових процеса могуће је

аутоматизовати и инфраструктуру што је најпопуларнија особина микросервисног приступа. Инфраструктура постаје сервис која је одговорност окружења за извршавање [1]. Окружење за извршавање најчешће користи технологију за контејнеризацију сервиса да би повећало ниво аутономије микросервиса [3].

Крајња функционалност система се остварује међусобном комуникацијом више микросервиса од којих је сваки најчешће имплементиран као процес на засебном хардверском чвору. Комуникација се најчешће изводи на релативно једноставан начин употребом HTTP захтјев-одговор механизма у REST стилу или асинхроне размјене порука [18]. Међусобна комуникација више микросервиса за извођење једне функционалности уводи додатни *overhead* на перформансе система стога је потребно минимизовати учесталост комуникације оваквог система.

Због већег нивоа дистрибуираности веће су шансе да не буду сви сервиси доступни у сваком тренутку, стога микросервисни приступ заговара да се адекватним дизајном система постигне да он настави да функционише са прихватањем отказа компоненти, али и да аутоматизује опоравак компоненти које откажу, најчешће користећи CI/CD. Ово значи да је за правилно функционисање микросервисних система правилан надзор од критичног значаја [1].

Микросервисни систем је фино гранулисани систем чије компоненте чини скуп независних сервиса те као такав пружа већу флексибилност за конфигурисање логике за скалабилност, доступност, перзистенцију, надзор и сигурност за сваки сервис посебно [3]. Додатно, сваки сервис може бити развијан кориштењем међусобно различитих технологија и од стране различитих тимова. Микросервис се третира као производ за чији је цијели животни циклус, укључујући и употребу у продукцији, задужен исти тим [1].

Микросервиси нису фокусирани на стандардизацију тога које развојне оквире и окружења користити. Нагласак је на прављењу корисних компоненти које након тога могу користити и други корисници и компоненте. Одабир конкретне технологије није од суштинског значаја јер постоји стабилна дефиниција сервиса која омогућава поновну употребу [1].

Да би се постигла висока аутономија микросервиса овај приступ користи децентрализовано управљање подацима које подразумијева да сваки микросервис брине о својим подацима и да не постоји централна база података [18]. Ово уједно значи и да сваки микросервис може имати другачији концептуални модел проблема који се рјешава његовом употребом у односу на друге микросервисе. Ако су једном сервису потребни подаци другог сервиса, он може да им приступи путем интерфејса другог сервиса. Као посљедица оваквог дизајна јавља се проблем како очувати конзистентност података који су дијелови више различитих сервиса. У ову сврху се могу користити дистрибуиране трансакције које су познате по томе што их је тешко имплементирати. Стога, микросервисни приступ прихвата чињеницу да ће конзистентност можда некада бити нарушена или да ће она бити достигнута тек у некој

будућности. Самим тим, нису сви проблеми погодни за вид реализације употребом микросервиса [1].

Пошто је микросервис усмјерен на пословне намјене не постоји гаранција да ће он бити нити стална нити непромјењива компонента. Микросервисни приступ прихвата особину еволутивности дизајна што значи да услијед промјене пословних захтјева сваки сервис може бити измијењен, потпуно замијењен или интегрисан у други сервис [1]. Најважнији циљ је испунити пословне захтјеве а дизајн компоненти је тај који му се прилагођава.

Кориштење микросервисне архитектуре информационог система пружа организацијама дугорочну агилност јер је лакше одржавати комплексне, широко-опсежне и системе који захтијевају високу скалабилност. Начин на који се ова агилност постиже је да се креирају апликације базиране на скупу независно испоручљивих сервиса с тим да сваки сервис има аутономни животни циклус [10]. Микросервиси превазилазе неке од недостатака SOA приступа да би прилагодили софтвер модерним пословним окружењима базираним на *cloud*-у [20].

2.3.1. Моделовање микросервиса

Микросервиси се моделују тако да буду концентрисани на пословне функције превазилазећи тако границе класичних слојева или архитектура [1]. Циљ је постићи високу кохезију пословних функционалности умјесто постизања високе кохезије техничких функционалности. Кохезија микросервиса је мјера сличности функционалности унутар његових граница. Функционалности су сличне онолико колико промјена једне функционалности изискује промјену других функционалности. Високом кохезијом се ефективно постиже да измјена, додавање или уклањање пословне функционалности не захтјева измјену више компонената система већ само оне којој она припада [10].

У пословним окружењима је популарно да се микросервиси моделују употребом једне од техника из доменом вођеног дизајна (DDD), званом *context mapping*. Ова техника рашчлањује доменски модел апликације и њему припадајуће податке на више доменских модела тако што идентификује контексте и њихове границе (*bounded context*). Ограничени контекст може бити скупина података који имају јаку кохезију базирану на већем броју међусобних зависности. Један доменски ентитет може бити дио више контекста с тим да сваки контекст може да их именује различитим пословним терминима и чува само себи релевантне податке тог ентитета (нпр. корисник може бити купац у једном контексту а продавац у другом). Један контекст се може употријебити за моделовање једног микросервиса јер идентификује његове доменске границе [10].

Циљ дизајна микросервиса није остварити максимално могућу грануларну сепарацију, иако би требало да постоји тежња за дизајном што мањих микросервиса могуће вођена доменским знањем. Како пројектант више упознаје домен проблема тако би требало да итеративно адаптира величину микросервиса. Оно што је добар

показатељ да је величина микросервиса адекватна је постојање малог броја јаких веза између њих без потребе за честом комуникацијом те непостојање потребе за честом агрегацијом података из различитих микросервиса приликом извршавања типичних операција апликације [10].

2.3.2. Дизајн приступне тачке микросервисном систему

Клијентске апликације могу приступити микросервисима или директним позивањем сваког сервиса или путем API *gateway*-а. Директан приступ може функционисати за системе мањег опсега, али у сложеним апликацијама доводи до вишеструких захтјева клијената ка различитим сервисима, веће латенције, повећане сложености на страни клијента, дуплираних функционалности (као што су аутентификација, логовање и кеширање), некомпатибилности протокола и јаке спреге између клијента и интерних сервиса.

API *gateway* рјешава ове проблеме тако што има улогу јединствене приступне тачке систему. Он пружа реверзно проксирање, агрегацију захтева (комбиновање више позива сервиса у један захтјев клијента) и функционалности као што су аутентификација, ауторизација, TLS терминација, логовање и кеширање. Ово смањује сложеност клијента и побољшава перформансе јер се користе сервер-сервер мреже које имају мању латенцију од клијент-сервер мрежа.

Међутим, један API *gateway* може постати превише сложен, чврсто повезан са сервисима и јединствена тачка отказа. Да би се то избјегло, препоручује се коришћење више мањих *gateway*-а на основу типа клијента, функционалности или пословних граница. Такође они се могу дизајнирати да имају високу скалабилност, отпорност и толеранцију на грешке тако да кварови у појединачним микросервисима не доведу до отказа цијелог система.

2.3.3. Комуникација микросервиса

За разлику од SOA приступа, микросервисни приступ тежи да се сва комплексна логика налази унутар сервиса а да се комуникација између њих изводи релативно једноставно. Због тога микросервиси, умјесто комплексних протокола из WS-* фамилије стандарда, користе комуникацију базирану на HTTP-у или на једноставној размјени порука. Такође, умјесто кориштења централизованог оркестратора за координацију пословних процеса, микросервиси су најчешће подвргнути кореографији [10].

Генерално, врсте комуникације у дистрибуираним системима имају сљедеће димензије [10]:

- Синхрона или асинхрона:
    1. Синхрона комуникација подразумијева да клијент шаље захтјев серверу чији одговор чека у блокираном (синхроно) или у неблокираном стању (асинхроно). Ово стање је независно у

односу на синхрону или асинхрону природу комуникације. Синхрона природа комуникације подразумијева да клијент очекује одговор посланог захтјева да би наставио своје исправно извршавање, све вријеме трошећи одређену количину меморијских и процесорских ресурса.

2. Асинхрона комуникација подразумијева да клијент пошаље поруку серверу и не чека његов одговор. Ако су клијенту потребне неке информације од сервера везане за ову поруку, потребна је додатна експлицитна комуникација. Да би оваква комуникација била ефикасна потребно је да постоји додатна инфраструктурна компонента и то најчешће брокер порука.

- Само један или произвољан број прималаца:

  1. Комуникација може бити таква да клијент шаље захтјев који ће бити процесиран само од једног примаоца. Ако клијент треба да пошаље већу количину података ка више примаоца, може да се доведе у ситуацију да нема довољно ресурса јер сваку ову акцију мора да уради засебно [1].

  2. Комуникација може да подразумијева да се захтјев или порука шаљу ка произвољном броју примаоца. Оваква комуникација је најчешће асинхрона и користи *publish-subscribe* модел.

Микросервисна апликација често користи комбинацију ових врста комуникација у зависности од захтјева система. Најчешће кориштена синхрона комуникација намијењена једном примаоцу је базирана на HTTP захтјев-одговор механизму [10]. Да би систем који пружа функционалности путем HTTP-а био сигуран, модуларан и скалабилан користи се API архитектура која се може дизајнирати са сљедећим парадигмама [9]:

- REST – Дизајн који је базиран на ресурсима и има фиксну структуру дефинисану од стране сервера. Клијент мора да поштује ову структуру која је релативно лака за кориштење али може да смањи ефикасност комуникације јер је приликом слања упита понекад потребно мање (*over-fetching* ситуација) или више (*under-fetching* ситуација) података о доменским ентитетима од тога колико сама структура у виду ресурса то омогућава. За постизање веће ефикасности потребно је омогућити грануларан приступ дефинисањем већег броја *endpoint*-а што отежава одржавање кода. Овај дизајн је веома погодан за мање апликације које не баратају са комплексним подацима потенцијално из више извора података.

- GraphQL – Такође је базиран на ресурсима с тим да има за циљ да побољша ефикасност упита у односу на REST. Ово постиже тако што користи *query* језик да спецификује како клијент захтјева податке од

сервера. Овај језик омогућава клијенту захтијевање или модификацију само оних података који су му потребни. Ово је могуће јер GraphQL *runtime* на серверској страни процесира упит који уједно мора бити у складу са уговором. За разлику од REST-а, GraphQL стриктно захтјева кориштење уговора у виду шеме која дефинише ресурсе и њихове *resolver* функције. Ове функције омогућавају добављање података сваког поља ресурса из произвољног извора података. Осим тога, GraphQL омогућава извршавање више акција над више ресурса једним мрежним позивом. Овај дизајн је погодан када су потребне високе перформансе комуникације и флексибилност кориштења API-ја с тим да се повећава комплексност серверског дијела апликације.

- gRPC – Такође користи захтјев-одговор механизам с тим што је базиран на HTTP/2 протоколу па омогућава да клијент пошаље више захтјева серверу путем једне конекције и добије више одговора путем исте. Користи *Remote Procedure Call* модел тако да се функционалности система извршавају тако што клијент позива удаљени сервис на серверској страни као да се ради о локалном сервису. Ово значи да је комплетна основа HTTP комуникације апстрахована. gRPC ствара већу спрегу између компонената од REST-а јер захтјева да и клијент и сервер имају приступ *Protobuf* датотеци која представља уговор којим се дефинишу сервиси, структуре порука итд. gRPC API је изузетно погодан за међусобну комуникацију интерних микросервиса који захтијевају високе перформансе и који размјењују велике количине података.

Синхрона захтјев-одговор комуникација је изузетно погодна када *frontend* клијент шаље упите за подацима за UI приказ или извршава операције чији резултат очекује у релативно кратком року. Због тога се ова врста комуникације најчешће користи на релацији *frontend* - микросервиси или, ако постоји API *gateway*, *frontend* - *gateway*. Ако клијент треба да иницира акције које се на *backend*-у извршавају дужи временски период може се користити WebSocket протокол. Ако апликација захтјева ажурирања у реалном времену може се, осим *WebSocket*-а, користити и SSE API.

Такође, синхрону врсту комуникације путем REST или GraphQL API-ја могу да користе и интерни микросервиси када међусобно комуницирају с тим да је потребно минимизовати ову интерну комуникацију да се не би нарушиле перформансе и повећала комплексност система. Сваки мрежни позив потребан за извршавање комплексног пословног процеса има латенцију, која у потенцијално учесталој микросервисној комуникацији може да има кумулативни ефекат [19]. Крајњи ефекат је нарушено корисничко искуство спорим одзивом. Ипак, често је потребно интегрисати микросервисе [10].

Међутим, у системима који захтијевају високу скалабилност синхрона комуникација може да има велики утицај јер HTTP *runtime* зависност клијента и сервера чини то да отказ *downstream* микросервиса може да наруши аутономију и

смањи перформансе *upstream* микросервиса или у најгорем случају доведе до каскадног отказивања више микросервиса [10].

Наиме, сваки микросервис треба да има аутономију извршавања и као такав, да буде доступан конзументима независно од тога да ли *downstream* сервиси са којима комуницира нису доступни. Ако се приликом извршавања крајње функционалности микросервиси ослањају на синхроне позиве може да се створи дугачак захтјев-одговор ланац који обухвата секвенцу више сервиса. Ако један сервис откаже или има слабије перформансе, што је неизвјесно у дистрибуираном систему, и други *upstream* сервиси у овом ланцу могу да имају исте проблеме. На примјер, пораст броја HTTP захтјева може да учини да микросервис потроши све ресурсе и тако постане недоступан ако један од сервиса од којих синхроно зависи није доступан или има слабе перформансе. Ово нарушава микросервисни принцип да систем буде дизајниран тако да настави да функционише и у случају парцијалног отказа у коликој год мјери је то могуће [10].

Један од начина рјешавања овог проблема је да *upstream* сервиси буду дизајнирани дефанзивно ради повећања отпорности од отказа *downstream* сервиса (нпр. употребом *circuit breaker*-а, *timeout*-а, враћања парцијалног или подразумијеваног одговора). Овакав дизајн се поготово користи код API *gateway*-а јер и он сам подлијеже истим проблемима везаним за синхрону зависност с тим да је он и централно мјесто кроз које пролази сав HTTP саобраћај према интерним микросервисима и самим тим захтјева још опрезније мјере.

Независно од ових проблема, кориштење HTTP-а јача спрегу између пословно вођених микросервиса који, у идеалном случају, не би требало да знају за постојање других пословних микросервиса. HTTP захтјева од клијента да зна адресу одредишног сервера или да одржава листу више заинтересованих сервера приликом слања података. Што компоненте више знају за постојање других компонената то се повећава комплексност система и отежава његово проширивање додавањем нових компонената [10].

Ако сви ови проблеми отежавају или потпуно нарушавају испуњавање захтјева конкретног система постоји опција да се комуникација између микросервиса одвија асинхроно. Ако микросервис треба да иницира акцију у другом сервису без да је моментално заинтересован за њен исход, то може да учини асинхроно. Ако су му пак потребни подаци другог сервиса, може да користи приступ у којем он потребне податке већ треба да има доступне унутар свог изворишта података. Ово се постиже техникама репликације и пропагације свих потребних података из осталих сервиса у сам циљни сервис и прије него што их он затреба. У ове сврхе се користи *eventual consistency* приступ и архитектура вођена догађајима [10].

Асинхрона комуникација елиминише потребу да компоненте знају за постојање других компонената употребом додатне компоненте и механизма просљеђивања порука. Микросервис шаље команду или захтјев другом сервису у виду поруке. Ако је потребно да сервис одговори клијенту, то врши слањем друге поруке. Клијент не очекује да одговор добије одмах нити да ће га уопште добити [10]. Најчешћа

инфраструктура за ову врсту комуникације је *lightweight* брокер порука који представља програм или модул који посредује између микросервиса који размјењују поруке, тако што преводи поруку из формалног протокола пошиљаоца у формални протокол примаоца. Осим тога, овај програм обавља и друге функционалности као што су валидација и усмјеравање порука [1]. Не садржи никакву пословну логику тако да одржава комуникацију једноставном док је и даље сва комплексна логика у сервисима који одлучују које поруке желе да продукују и конзумирају. Најчешће кориштени асинхрони протоколи за комуникацију порукама дистрибуираних система су AMQP и *Kafka*.

Ако се интерни микросервиси дизајнирају да искључиво комуницирају асинхроно тада *frontend* клијент може да синхроно иницира акцију коју затим API *gateway* асинхроно шаље микросервисима. Затим клијент може да чека на одговор употребом *WebSocket*-а ка *gateway*-у или да врши HTTP *polling* на специјалне *endpoint*-е за провјеру статуса резултата. Сервис који пружа ове *endpointe* асинхроно комуницира са микросервисима очекујући резултат. Ово све уводи додатну комплексност у API *gateway* и *frontend* компоненту.

За комуникацију један-на-један брокери порука користе ред порука у који *producer*-и шаљу поруке од којих ће свака бити испоручена само једном конзументу. Порука је у овом случају најчешће команда која енкапсулира намјеру *producer*-а за извршавање одређене акције на страни конзумента.

Постојање додатне компоненте у виду брокера порука омогућава и слање порука ка више конзумената. У ове сврхе се користи *publish-subscribe* механизам тако што брокер порука имплементира *event bus* и пружа API за претплаћивање и објаву догађаја. *Event bus* омогућава архитектуру вођену догађајима која подразумијева софтверску архитектуру и модел дизајна апликација која се заснива на прављењу, дистрибуцији, обради и перзистенцији догађаја. Догађај представља било какво значајно дешавање или промјену стања система [1]. Микросервиси генеришу поруке о догађајима на чији пријем може бити претплаћено више других микросервиса.

Неке од предности које пружа просљеђивање порука у односу синхрону захтјев-одговор комуникацију су [8]:

- Слаба спрега – Због постојања додатне посредничке компоненте, *producer* порука не мора да зна за локацију, њихов број, њихове функције нити да ли су конзументи активни или нису [1]. И даље је потребно да *producer* и конзумент користе исти уговор порука.
- Флексибилност и поновна употребљивост – Систем је могуће еволуирати и надограђивати без измјене иједне постојеће компоненте [1].
- Отпорност и изолација отказа – Ако конзумент није доступан, *producer* и даље може да пошаље поруке које заједно са каналима којима се преносе могу бити *durable* што значи да ће поруке, које није могуће одмах испоручити, бити сачуване. Додатно, поруке могу бити *persistent* што значи да ће поруке које нису испоручене преживјети рестарт брокера [1]. Ово је итекако погодно за микросервисну архитектуру јер сваки

микросервис има свој животни циклус независан од других микросервиса [8].
- Непостојање блокирања – *Producer*-и и конзументи могу да шаљу и процесирају поруке у произвољним временским интервалима.

Дизајнирањем система као скуп слабо спрегнутих дискретних сервиса који комуницирају помоћу догађаја повећава се флексибилност и скалабилност система по цијену увођења додатних инфраструктурних компонената.

2.3.4. Микросервисно управљање подацима

Микросервисни приступ захтијева управљање подацима на децентрализован начин с циљем постизања што веће аутономије микросервиса. Сваки микросервис је власник својих доменских података које чува унутар своје базе података што се разликује од SOA приступа у којем сервиси могу користити дијељене базе података [10]. Сваки микросервис треба да је задужен за своје стање а стањима других сервиса приступа путем добро дефинисаног интерфејса [1].

Енкапсулацијом података смањује се спрега између микросервиса и повећава њихова могућност да независно еволуирају. Ако би више сервиса користило исту базу података, промјена шеме тих података би захтијевала координисано ажурирање свих сервиса што би нарушило њихов аутономни животни циклус. Међутим, због тога што су сада подаци дистрибуирани на више микросервиса није тривијално извршити ACID трансакцију над њима као што је то могуће извести на нивоу једне базе података кориштењем система за управљање базама података [10]. ACID трансакција је она трансакција која постиже највећу поузданост и интегритет података испуњавањем сљедећих својстава: атомичност, конзистентност, изолација и трајност [11].

Због тога што се ради о дистрибуираном систему, подразумијева се да ће дијелови микросервисног система отказати. Стога је потребно прихватити толеранцију на отказе те по CAP теореми бирати између доступности и јаке ACID конзистентности. Већина микросервисних апликација захтјева доступност и високу скалабилност али не и јаку конзистентност. Ово значи да ако је дошло до парцијалног отказа сваки захтјев треба да добије одговор који не мора да садржи најажурније податке. Ипак, одржавање конзистентности података између микросервиса мора да буде остварено. У те сврхе се користи *eventual consistency* приступ који подразумијева прихватање чињенице да ће конзистентност података можда некада бити нарушена или да ће бити постигнута тек у будућности [10].

С обзиром на то да одређени корисници микросервисног система могу очекивати да раде се најажурнијим подацима, што не мора бити случај када се користи *eventual consistency* приступ, треба дизајнирати систем тако да он буде експлицитан по питању да ли се користи овај приступ да би се одржао висок ниво корисничког искуства [10].

Такође, изазов одржавања конзистентности је везан и за питање како пропагирати промјене података на више микросервиса када је потребно да ти подаци у

њима буду редундантни. Кориштењем одређених техника може се постићи *eventual consistency* и оне се најчешће базирају на употреби архитектуре вођене догађајима [10].

Једна од ових техника је кориштење интеграционих догађаја који представља догађај који микросервис објављује након што се десила одређена промјена унутар његовог домена као што је ажурирање доменских ентитета, из разлога што ова промјена може бити од интереса другим микросервисима. Други микросервиси се могу претплатити на овај догађај, након чега могу асинхроно добити поруку о њему те га искористити за ажурирање својих доменских ентитета и евентуално објавити нове интеграционе догађаје. Ова динамика чини суштину *eventual consistency* приступа те се користи за одржавање конзистентности података када пословни процес обухвата више микросервиса [10].

Микросервис који генерише интеграциони догађај треба да изврши ову акцију атомично са ажурирањем своје базе података јер се систем може довести у трајно неконзистентно стање услијед различитих узрока као што су отказ других микросервиса или недоступности *cloud* базе података [10].

2.3.5. Отпорност и висока доступност микросервиса

При дизајну микросервисног система мора се рачунати на неизбјежне отказе како би се постигле отпорност и висока доступност. Циљ није елиминисати отказе, већ их прихватити и омогућити систему да се након њих брзо врати у потпуно функционално стање. Парцијални откази настају када један или више сервиса откажу због пада сервера, проблема са мрежом или спорих одговора. Ако клијентски сервиси нису правилно дизајнирани (нпр. користе синхроне HTTP позиве без *timeout*-а), може доћи до исцрпљивања ресурса и каскадног отказивања више сервиса.

Ефекти ових проблема ограничавају се механизмима као што су *retries* са експоненцијалним одлагањем, обавезни *timeout*-ови, *circuit breaker* шаблон (привремено обустављање захтјева након више неуспјелих захтјева) и враћање подразумијеваних или кешираних одговора када сервис није доступан. Употреба брокера порука као што су *RabbitMQ, Apache ActiveMQ* и *Azure Service Bus* додатно повећава поузданост кроз *store-and-forward* механизам, трајност порука, аутоматско поновно слање порука и контролу дупликата.

Поред клијентске улоге, неопходно је обезбиједити и отпорност сервиса као сервера. То подразумијева могућност рестартовања у било ком тренутку без губитка или нарушавања конзистентности података, као и механизме за детекцију отказа и аутоматски рестарт, често уз помоћ оркестратора и *Service Mesh* технологија. Тако се обезбјеђује континуирана доступност и стабилност система чак и током отказа или надоградњи.

2.3.6. Оркестратори и контејнеризација микросервиса

Да би се развојни тимови растеретили рјешавања комплексних инфраструктурних проблема и тако им се омогућило да се посвете рјешавању

конкретних пословних проблема и развоју микросервисних апликација, развијене су микросервисно-оријентисане платформе које се називају оркестратори или сервисни кластери. С обзиром на то да се микросервиси најчешће извршавају у оквиру контејнера који могу да се извршавају на великом броју различитих *host*-ова, од критичног значаја је да постоји ефикасан начин управљања кластерима које чине ови *host*-ови. Оркестратори пружају управо ову функционалност тако што апстрахују комплексност нижих платформи. Имају за циљ да ријеше проблеме који се јављају током развоја и извршавања сервиса те да на ефикасан начин омогуће сервисима употребу инфраструктурних ресурса. Конкретни задаци везани за микросервисе које рјешавају су: управљање њиховим животним циклусом, независно скалирање и ажурирање, висока доступност, *load-balancing*, рутирање, адресибилност, отпорност, *health*, дијагностика итд. [10].

Оркестратори користе контејнеризацију која представља процес *deployment*-а софтвера који пакује код апликације са свим датотекама и библиотекама који су јој потребни за извршавање на произвољном хардверу и оперативном систему. Овако настао софтверски пакет се назива слика контејнера а његова *runtime* инстанца је контејнер. Контејнер је портабилна *lightweight* софтверска компонента која омогућава апликацији да се извршава изоловано од других апликација. Ефикасним извршавањем сваког контејнера управља контејнер *runtime* који се извршава поврх оперативног система. Ова портабилност, изолација и ефикасност су пожељне особине за микросервисни систем. Програмски код микросервиса написан на један начин може да се извршава у различитим окружењима без потребе за било каквим промјенама. Изолација омогућава већу аутономију јер микросервис мање зависи од спољашњих фактора. Ефикасност води ка већој скалабилности јер оркестратор може брже стартати знатан број инстанци микросервиса било то на једном или више *host*-ова [13].

Употреба контејнера омогућава лакшу аутоматизацију и краће вријеме животног циклуса софтвера. Контејнери елиминишу конфликте у употреби апликације између развојног, тестног и продукционог окружења јер слика контејнера представља усаглашену имутабилну јединицу управљања апликацијом и у свакој овој фази се понаша идентично [15]. На овај начин контејнери омогућавају логичку подјелу посла који може бити организован у више развојних, оперативних и *DevOps* тимова. Често се ови тимови преклапају или се чак групишу у један *DevOps* тим који ради са цијелим животним циклусом софтвера [14].

Ова подјела омогућава да један тим (развојни или *DevOps*) има власништво над садржајем контејнера, његовим сервисима и апликацијама, зависностима, развојним оквирима, и над тиме како сервиси унутар контејнера интерагују у сврху компоновања крајње апликације. С друге стране, други тим (оперативни или *DevOps*) користи слике контејнера, *manifest* датотеке и оркестратор да се фокусирају на управљање продукционим окружењем, инфраструктуром, скалабилношћу, надзором те утврђивањем исправне испоруке апликације крајњим корисницима [15].

2.3.7. Service Mesh

Service Mesh је приступ који рјешава проблеме дистрибуиране комуникације у микросервисним системима тако што издваја логику међусервисне комуникације у посебан инфраструктурни слој. Тај слој се састоји од *data plane*-а (мрежни прокси сервиси имплементирани као *sidecar* контејнери уз сваки микросервис) и *control plane*-а (централни управљачки и конфигурациони слој). Апликациони контејнери не морају бити мијењани нити су свјесни постојања овог слоја, чиме се задржава њихова аутономија.

Data plane обезбјеђује функционалности као што су откривање сервиса путем сервисног регистра, *load balancing*, раздвајање и дуплицирање саобраћаја, увођење нових верзија методом канаринца, сигурност (укључујући mTLS, аутентификацију и ауторизацију), надзор (метрике, логови, дистрибуирани *tracing*) и механизме отпорности попут *circuit breaking*-а и поновног слања захтјева. *Control plane* управља конфигурацијом, аутоматски открива и уклања сервисе, прикупља телеметрију и динамички дистрибуира правила *proxy* сервисима без рестартовања система. Најпознатије имплементације су *Istio* и *Linkerd*, које су намијењене да раде у оквиру *Kubernetes* окружења.

Иако *Service Mesh* растерећује микросервисе од сложене комуникационе логике, његове мане су повећана инфраструктурна комплексност, потреба за додатном конфигурацијом и увођење *control plane*-а као потенцијалне критичне тачке отказа која захтијева посебно скалирање и заштиту.

2.3.8. DevOps

DevOps је комбинација филозофије, пракси и алата која има за циљ да организацијама омогући да скрате животни циклус софтвера да би брже испоручили исти високог квалитета у односу на традиционалне методе развоја софтвера. *DevOps* модел подразумијева да не постоји јасна граница између развојних и оперативних тимова и да су они често спојени у један *DevOps* тим. Ови тимови користе праксе и алате за аутоматизацију процеса развоја софтвера и управљања инфраструктуром што им омогућава да оперишу и еволуирају апликације на бржи и поузданији начин [14].

Када се *DevOps* користи са микросервисима, *DevOps* се рашчлањује на више *DevOps* тимова гдје сваки тим има независност у односу на друге јер има власништво над својим сервисом и засебно управља његовим животним циклусом. Један тим независно испоручује код и снабдијева инфраструктуру свог сервиса. Такође, ажурирања микросервисног кода су мања те заједно са CI/CD аутоматизацијом омогућавају већу фреквентност и темпо објављивања апликације. Фреквентна и мања ажурирања смањују ризик објављивања јер тимови могу лакше да пронађу багове идентификацијом посљедњег ажурирања. Сама испорука је поузданија јер се свака промјена аутоматски тестира кроз CI/CD. Додатно, праксе надзора и логовања омогућавају информисање о перформансама у реалном времену и брзо уочавање грешака [14].

Унутар *DevOps* окружења тимови константно колаборирају и комбинују своје радне токове путем успостављене контејнерске платформе. *DevOps* тимови интегрирају оперативне активности са програмерским алатима и објављивањем кода да би оптимизовали продуктивност програмера и поузданост IT оператора [16]. QA тимови и сигурносни тимови такође могу бити учесници у константној колаборацији [14]. Штавише, често пословна окружења реструктуришу своје организације тако да оне представљају људе и процесе који су у складу са контејнеризованим радним током [10].

DevOps омогућава и руковање инфраструктурним и развојним процесима на већем нивоу скалабилности. Нпр. честа пракса је кориштење *Infrastructure as a Code* приступа (IaC) који омогућава програмско, репетитивно и ефикасно управљање развојним, тестним и продукционим окружењима. Аутоматизација инфраструктуре олакшава да рачунарски ресурси буду скалирани и респонзивни на фреквентне промјене саобраћаја [14]. Овај приступ се интегрира у CI/CD *pipeline* да би се инфраструктура могла прилагођавати софтверу док он пролази кроз процесе *build*-а и испоруке [16].

Континуална интеграција (CI) је пракса у којој програмери редовно интегришу код у централни репозиторијум, након чега интеграциони сервиси аутоматски покрећу *build* и јединичне тестове у циљу раног проналаска грешака. Осим компоненте аутоматизације (*build* сервис) CI подразумијева и културолошку компоненту (нпр. створити навику фреквентне интеграције кода). Главни циљеви CI праксе су: проналазак багова прије него што они прерасту у веће проблеме, повећање квалитета софтвера и смањивање времена валидације и испоруке ажурираних верзија софтвера [14].

Континуална испорука (CD) је пракса која се надовезује на CI и подразумијева да се, након *build* фазе, промјене кода аутоматски испоручују у тестно и/или продукционо окружење. Ово тестирање је обухватније од јединичног тестирања и омогућава верификацију ажурирања апликације у више димензија прије него што оно буде испоручено у продукцију. Оно може укључивати UI тестирање, тестирање оптерећења, интеграционо тестирање, API тестирање итд. У ове сврхе је веома погодна употреба IaC приступа јер оно омогућава једноставну и исплативу аутоматизацију креирања и репликације различитих окружења за тестирање. Коначно одобрење о објављивању мануелно доноси одређени ауторитет. Ако се користи континуално објављивање (*Continuous Deployment*) тада се објављивање врши аутоматизовано без потребе за мануелним одобрењем [14].

Уобичајени микросервисни *DevOps* радни ток укључује сљедеће акције [15]:

1. Програмер пише, покреће, дебагује, *build*-а и тестира микросервис у свом локалном репозиторијуму.
2. Затим, развојни тимови врше *push* изворног кода на репозиторијум система за управљање изворним кодом као што је *Git*.

3. На основу изворног кода, *Dockerfile-а* и додатних инструкција, екстерни *build* сервиси као што су *GitHub* и *Azure DevOps Services* аутоматски покрећу акције дефинисане од стране развојних или *DevOps* тимова као дио CI *pipeline-а*. Инстанцира се *Docker* контејнер унутар удаљеног *Docker host*-а те се врше *build* и тестови. Затим ови сервиси врше објаву изграђених *Docker* слика у репозиторијум артефаката као што је *Docker* регистар те тако успјешно завршавају интеграцију кода. *Docker* регистар спаја CI и CD *pipeline*.

4. Овако генерисане *Docker* слике могу аутоматски бити објављене у више окружења (продукционо, тестно, *staging*) као дио CD *pipeline-а*. За објављивање апликације у дистрибуиране *Kubernetes* кластере или примјену IaC приступа, *DevOps* тимови користе специјализоване *deployment tasks* које пружају сервиси као што су *Azure DevOps Services* и *GitHub Actions*.

5. Оперативни тимови управљају апликацијом и њеним извршавањем у скалабилном, продукционом окружењу са високом доступношћу користећи оркестраторе кластера.

6. Оперативни тимови врше надзор и поврат информација развојним тимовима о томе како апликација може бити побољшана употребом различитих *cloud* базираних, *Kubernetes* нативних или *Service Mesh* алата.

Овај цјелокупни процес аутоматизује цијели животни циклус софтвера и може се итерирати неограничен број пута [15]. *DevOps* у комбинацији са микросервисима креира робусни *framework* за испоруку квалитетних апликација на брз и поуздан начин [16].

2.3.9. Предности и недостаци микросервисног приступа

Бенефити које микросервисни приступ пружа су [10]:

- Сваки микросервис је релативно малог опсега и лако је управљати њиме и еволуирати га из сљедећих разлога:

    1. Развојни тимови могу лако да разумију и брзо започну своје задатке.

    2. Контејнери се покрећу брзо што повећава продуктивност тимова.

    3. Интегрисана развојна окружења брзо учитавају мале пројекте повећавајући тако продуктивност тимова.

    4. Сваки микросервис може бити дизајниран, развијан, и објављен независно од других микросервиса. Ово омогућава већу агилност због тога што је лакше фреквентно објављивати нове верзије сервиса за разлику од другачијих приступа.

- Независно скалирање – Због тога што микросервиси имају ограничен опсег функционалности они често могу бити подвргнути варијабилним оптерећењима. У оваквим ситуацијама, због тога што су микросервиси аутономни није потребно скалирати цијелу апликацију, већ само одговарајући сервис. С обзиром на то да су микросервиси оријентисани ка флексибилности, често се користе оркестратори микросервиса који подржавају аутоматско скалирање сваког сервиса на основу оптерећења.
- Подјела одговорности између независних тимова – Сваки тим има власништво над једним сервисом и одговоран је за његов развој, управљање, објављивање и скалирање независно од других тимова. Овиме се смањује потреба за међутимском координацијом што доводи до уштеде времена и трошкова [3].
- Технолошка хетерогеност – Подржан је слободан избор технологије за развој сваког микросервиса због тога што постоји стабилна дефиниција сервиса која сакрива имплементационе детаље.
- Поновна употребљивост – Као и код SOA приступа, сваки микросервис може бити кориштен у сврхе различитих апликација.
- Робусност – Отказ једног сервиса не мора да доведе до отказа цијелог система већ само до деградације функционалности коју тај сервис пружа.
- Лакша испорука – Могуће је испоручити сервис или нову верзију сервиса без нарушавања рада остатка система.

С друге стране, кориштење микросервисног приступа ствара сљедеће мане и препреке [10]:

- Дистрибуирана апликација – Дистрибуирање апликације захтјева додатну комплексност приликом дизајна и развоја. Потребно је имплементирати међупроцесну комуникацију протоколима као што су HTTP или AMQP а они уводе додатну комплексност приликом тестирања и управљања изузецима. Осим тога, дистрибуирана апликација има већу латенцију па стога овај приступ није погодан за системе код којих је брз одзив од критичног значаја.
- Комплексност објављивања – Постићи високу скалабилност микросервисне апликације захтјева висок ниво комплексности везане за њено објављивање. Ако се не користе оркестратори, ова комплексност може захтијевати много већи напор током развоја него само имплементирање пословног дијела апликације.
- Атомичне трансакције – Постићи овакве трансакције између више микросервиса најчешће није могуће. Пословни захтјеви треба да су такви да омогућавају *eventual consistency* приступ између више микросервиса.

- Повећана глобална потрошња ресурса – Већи ниво грануларности и дистрибуираности у односу на монолитну апликацију захтијева више ресурса потребних серверима и *host*-овима као што су укупна меморија, дискови и мрежни ресурси. Ови додатни трошкови могу представљати проблем у почетку развоја и продукције микросервисне апликације, али с временом, могућност независног скалирања доноси организацијама већи профит него монолитна апликација која ствара дугорочне трошкове током њеног еволуирања.
- Проблеми у директној комуникацији клијента и микросервиса – Оваква комуникација може захтијевати слање већег броја захтјева од стране клијента што смањује ефикасност интеракције. Додатно, микросервиси могу користити протоколе који нису прилагођени вебу и *firewall*-има. Такође, рефакторисање микросервиса може нарушити компатибилност са клијентским апликацијама. Због свега овога у неким ситуацијама је препоручено користити неколико фино гранулисаних API gateway компоненти.
- Партиционисање на микросервисе – Партиционисати пословну апликацију на више аутономних микросервиса захтјева примјену додатних техника и приступа.

2.4. Микросервисни у поређењу са SOA приступом

SOA је намијењена за дистрибуиране пословне информационе системе, док за мање, пословно специфичне апликације, не пружа довољно флексибилности приликом скалирања. Разлози за ово су [20]:

- Постојање критичне тачке отказа у виду ESB-а.
- Употреба дијељених компонената што спречава индивидуално управљање сервисима.
- Сервиси пружају широк опсег функционалности што повећава вјероватноћу за отказивање цијелог пословног радног тока ако дође до отказа једног сервиса.

С друге стране, микросервисна архитектура пружа фино гранулисан начин креирања апликација у виду сервиса мањих функционалних опсега. Микросервисни приступ постиже ограничену или никакву зависност између индивидуалних сервиса. Сваки микросервис оперише независно да би пружио специфичну функционалност. Микросервиси не дијеле ресурсе као сервиси у SOA па је стога мања шанса за отказ цјелокупног система [20].

Код микросервиса је комуникација врло проста (REST, MQ, GraphQL) док је код сервиса понекад и врло комплексна (ESB) [1].

Иако микросервиси као и SOA, стављају нагласак на поновну употребљивост, још важнија особина је независност података сваког микросервиса. SOA сервиси ће често користити заједнички извор података док ће, супротно од тога, сваки микросервис имати свој извор података [20].

Микросервиси су фокусирани на *DevOps* приступ и CI/CD док је код SOA то слабије изражено [1]. SOA сервиси су донекле међусобно зависни па их је теже је испоручивати и објављивати. Такође, њихова већа међузависност и централизованост отежава контејнеризацију а самим тим и прилагођавање *cloud* окружењу [20].

Међутим, организације које имају *legacy* или самосталне пословне апликације могу да имају користи од SOA архитектуре. SOA поједностављује конвенционалне софтверске програме у мање, модуларне дијелове. Умјесто развоја преклапајућих и редундантних сервиса, развојни тимови могу користити постојеће SOA сервисе да имплементирају додатна пословна рјешења [20].

Микросервисна архитектура има већу подршку за агилне развојне тимове који могу вршити рапидне и инкрементaлне промјене без утицаја на стабилност апликације користећи CI/CD алате. Осим тога, микросервиси омогућавају аутоматско снабдијевање рачунарских ресурса и скалирање појединачних сервиса у реалном времену. Стога, компаније могу да искористе модерне *cloud* могућности и да са лакоћом објављују велики број микросервиса [20]. С друге стране, конзистентност података може бити нарушена или достигнута тек у некој будућности, тако да нису сви проблеми погодни за овакав вид реализације. Стога су микросервиси доминантно усмјерени на скалабилне веб апликације [1].

3. Преглед програмског језика Rust

Јавно први пут објављен 2012. године од стране *Mozilla* тима, Rust је програмски језик опште намјене, дизајниран тако да подстиче писање поузданог софтвера који је ефикасан по питању брзине извршавања и употребе меморије. Пошто гарантује високу сигурност меморије веома је погодан за *low-level* системско програмирање, док је у исто вријеме довољно експресиван и прилагођен за писање *backend* софтвера. Осим што постиже велику брзину извршавања, која се може поредити са брзином извршавања C/C++ програма, Rust омогућава и брзо писање кода због своје експресивности, помоћних алата и краћег времена дебаговања. Краће дебаговање и стабилнији програми се постижу употребом стриктног компајлера. Rust користи *ahead-of-time* компајлирање које производи самосталну бинарну датотеку која је специфична за циљну платформу. Велики број суптилних багова који су уобичајени за *low-level* код као што су багови везани за конкурентност и приступ нежељеној меморији компајлер ће препознати, указати на њих и спријечити компајлирање. Компајлер ово постиже употребом *ownership* система који представља модел управљања меморијом без употребе *garbage collector-a*. Заједно са системом типова он гарантује сигурност меморије. *Ownership* систем и многе друге карактеристике Rust-а су такозване *zero-cost* апстракције, што значи да оне немају негативан утицај на перформансе током извршавања с тим да одузимају вријеме током компајлирања услијед потребе за оптимизацијом кода [21].

Rust је статички типизиран језик гдје тип сваке варијабле мора бити познат за вријеме компајлирања чиме се постиже сигурност кориштења варијабли. Подразумијевано, варијабле су немутабилне након иницијализације. Ово је начин да се подстакне писање сигурног кода и лакше постигне конкурентност. Декларисање варијабле као немутабилне омогућава компајлеру да гарантује њену непромјењивост за све вријеме извршавања програма свим дијеловима кода који је користе као такву. С друге стране мутабилност варијабле потребно је експлицитно навести кључном ријечју *mut* [21].

Rust посједује концепт референци који омогућава начин да више дијелова кода приступа одређеним подацима без потребе за вишеструким копирањем тих података у меморији. Уобичајено, референце уносе комплексност у програме па стога Rust има специфичан приступ овом проблему који омогућава лак и сигуран начин кориштења референци и који је у вези са *ownership* системом [21].

3.1. Ownership систем

Умјесто кориштења *garbage collector*-а или експлицитног деалоцирања меморије Rust користи другачији приступ управљања меморијом који подразумијева употребу *ownership* система којег чине скуп правила које компајлер провјерава. Код неће бити компајлиран ако није у складу са сљедећим правилима [21]:

- Свака вриједност мора да има власника.
- У било којем тренутку извршавања може да постоји само један власник неке вриједности.
- Када власник изађе из опсега, вриједност ће бити одбачена.

Власник је најчешће варијабла а њен опсег је дио програма у којем је она валидна. Ако би Rust користио мануелно деалоцирање меморије са *heap-a*, могло би да дође до сљедећих ситуација: расипање меморије услијед заборавка деалоцирања, приступ невалидним подацима услијед прераног деалоцирања те ненамјерна инвалидација валидних података услијед вишеструког деалоцирања исте меморије. Да би ријешио ове проблеме Rust користи *ownership* правила и аутоматски деалоцира меморију позивом специјалне *drop* функције када варијабла која је власник података у тој меморији изађе из опсега. Овакво понашање има значајан утицај на начин писања Rust кода [21].

Приликом додјеле вриједности једне варијабле која садржи показивач на податке на *heap*-у (нпр. *smart pointer* као што је *String*) другој, аутоматски се врши *shallow copy* ове вриједности. Ово би довело до ситуације да двије варијабле указују на исти податак на *heap*-у те стога, Rust од тренутка додјеле чини прву варијаблу невалидном у сврху спречавања двоструког деалоцирања меморије. Свако даље кориштење овакве варијабле изазваће компајлерску грешку. Овиме се постиже да *drop* функција буде позвана само за једног, посљедњег власника. Ова операција додјеле врши трансфер власништва над вриједношћу и назива се *move* операција. Ако се она врши над типовима фиксне величине (примитивни типови, низови), који су на *stack*-у, њихова вриједност ће бити копирана те неће доћи до инвалидације варијабле чија вриједност се додјељује. Ово за посљедицу има да Rust неће аутоматски вршити *deep copy* података тако да свако аутоматско копирање нема значајан утицај на *runtime* перформансе.

```
let s1 = String::from("Здраво");
let s2 = s1;     // s1 постаје невалидна варијабла
```

Слика 3.1 – Примјер *move* операције

Ако се врши преписивање вриједности варијабле новом вриједношћу, аутоматски ће бити позвана *drop* функција за стару вриједност. Ако је потребно извршити *deep copy*, потребно је то урадити експлицитним клонирањем. Такође, исти механизми копирања или трансфера власништва се дешавају и приликом слања аргумената функцији и враћања вриједности из функције [21].

Да би се избјегао учестали трансфер власништва над вриједношћу, Rust посједује механизам референци који омогућава кориштење вриједности неке варијабле без трансфера власништва. Референца је слична показивачу у смислу да представља меморијску адресу која се може искористити за приступ подацима смјештеним на тој адреси. За разлику од показивача, Rust референца гарантовано показује на валидну вриједност одређеног типа док год је у опсегу. Акција креирања референце се назива

посуђивање и врши се оператором *&*. Након посуђивања, подаци и даље остају у власништву изворне варијабле. Подразумијевано, референца се може користити само за читање вриједности а ако је потребно и мијењати посуђену вриједност потребно је креирати мутабилну референцу кориштењем оператора *&mut* [21].

Опсег референце почиње од тренутка када је она дефинисана до тренутка када је она посљедњи пут искориштена. Кориштење мутабилних референци је ограничено тиме да је дозвољено користити мутабилну референцу на неку вриједност у било ком тренутку само и ако само тај тренутак није у опсегу неке друге референце на исту варијаблу, била она мутабилна или немутабилна [21].

```
        let mut s = String::from("Здраво");
        let r1 = &mut s;      // почиње опсег референце r1
        let r2 = &mut s;      // почиње опсег референце r2
        println!("{}, {}", r1, r2);     // проблем, опсези мутабилних референци се
преклапају.
        // Завршава се опсег референце r2
        let r3 = &s;   // почиње опсег референце r3
        println!(“{}, {}”, r1, r3); // проблем, опсези мутабилне и немутабилне референце се
        // преклапају. Завршава се опсег референци r1 и r3
```

Слика 3.2 - Примјер кориштења више мутабилних и немутабилних референци на исту вриједност

Ово ограничење над употребом референци омогућава компајлеру да врши превенцију потенцијалних *data race* ситуација које подразумијевају да двије или више референци приступа истој вриједности на несинхронизован начин уз услов да бар једна од њих врши писање. Ове ситуације могу да проузрокују недефинисано понашање које је тешко уочити и исправити током извршавања. Такође, није дозвољено да тренутак кориштења немутабилне референце буде у опсегу друге мутабилне референце из разлога што дијелови кода који користе немутабилну референцу не очекују да се вриједност неочекивано промијени док је читају. С друге стране, више немутабилних референци је дозвољено користити у било ком тренутку јер оно не може да доведе до недефинисаног понашања. Компајлер препознаје опсеге референци и стога дозвољава сљедећу ситуацију:

```
        let mut s = String::from("Здраво");
        let r1 = &s;   // нема проблема
        let r2 = &s;   // нема проблема
        println!("{r1} i {r2}"); // завршава се опсег референци r1 и r2
        let r3 = &mut s;
        println!("{r3}");     // нема проблема
```

***Слика 3.4* - Примјер кориштења мутабилних и немутабилних референци без преклапања њихових опсега**

Још један проблем који се може јавити код употребе референци јесте *dangling reference* ситуација која настаје када се сачува референца на деалоцирану меморију. Овиме се даје недозвољен приступ деалоцираној меморији дијеловима кода који нису власници података у овој меморији. Rust гарантује да референца никад неће бити *dangling reference* јер компајлер неће дозволити да референца остане дуже у опсегу од података које референцира (слика 3.5) [21].

```
fn main() {
    let reference_to_nothing = dangle();
}
fn dangle() -> &String {
    let s = String::from("zdravo”);
    return &s;
}
```

Слика 3.5 - Примјер *dangling reference* ситуације

Због тога што је варијабла *s* дефинисана у опсегу функције *dangle*, приликом враћања из функције, биће позвана *drop* функција над њом што би учинило да повратна вриједност референцира деалоцирани дио меморије. Због тога компајлер не дозвољава враћање референце из функције на локално дефинисану варијаблу *s*.

Додатно, Rust посједује и *slice* концепт који представља тип динамичке величине који референцира секвенцу сусједних елемената унутар *String* типа, *string* литерала, низа, колекције итд. Као и референце на варијабле, не преузима власништво над вриједношћу те подлијеже свим осталим *ownership* правилима [21].

3.2. Типови

Rust омогућава дефинисање *custom* типова у облику структура (*struct*) и енумерација (*enum*) који се могу искористити за повећање сигурности програма јер компајлер врши строгу провјеру типова. Структура омогућава креирање типова који имају смисла у домену проблема. Групише повезане податке (поља) и њихова понашања (методе и функције) и даје им имена. Енумерације омогућавају дефинисање типа на начин да се енумеришу варијанте које он може да поприми. Вриједност *enum* типа може да буде само један од скупа дефинисаних вриједности. Дефинисање *enum-а* је слично дефинисању више структура с тим да су све варијанте груписане унутар једног *enum* типа. Ово омогућава дефинисање функција и метода над свим варијантама што не би било тривијално извести у случају више засебних структура. Препознавање које варијанте је инстанца неког *enum*-а се може извршити моћном конструкцијом контроле тока која се назива *match* [21].

Уопштеније, *match* конструкција омогућава поређење неке вриједности са низом *pattern*-а те извршавање оног дијела кода који одговара *pattern*-у са којим се вриједност подудара. Слична је условном изразу који се користи са кључном ријечју *if* у оквиру које је потребно да условни израз буде евалуиран у *boolean* вриједност, док у случају *match*-а условни израз може да буде било ког типа. *Pattern* може да се састоји из вриједности, назива варијабли, *wildcard* карактера итд. Велика употребљивост *match* конструкције произилази из високе експресивности *pattern*-а и чињенице да компајлер омогућава компајлирање само ако утврди да су све могуће вриједности *match* условног израза обрађене. Ова употребљивост се посебно огледа у специјалном *enum* типу *Option<T>* који описује уобичајени сценарио гдје вриједност може да постоји или да уопште не буде присутна [21].

3.2.1. Непостојање *null* вриједности

Rust не посједује *null* вриједност већ концепт непостојања вриједности уграђује у систем типова што омогућава компајлеру да захтјева да су случајеви постојања и непостојања вриједности увијек обрађени. Кориштење неке варијабле која нема вриједност под претпоставком да је вриједност увијек присутна доводи до *runtime* грешке у програму (нпр. *null dereferencing*). Да би ово спријечио, Rust користи сљедећи *enum* дефинисан у стандардној библиотеци:

```
enum Option<T> {
    None,
    Some(T),
}
```

Слика 3.6 - Option<T> enum

T је генерички параметар и омогућава креирање *Some(T)* варијанте *Option enum-a* која садржи произвољан тип *T*. Због тога што су *Option<T>* и *T* различити типови није могуће користити вриједност варијабле типа *T*, која не мора бити присутна, без претходне обраде *Option<T> enum-a* и то најчешће користећи *match* конструкцију.

```
let numbers = vec![10, 20, 30];

// приступ 4. елементу вектора који не постоји
let value = match numbers.get(3) {
     Some(val) => val,
     None => 0,
 }
// варијабла value од овог тренутка сигурно садржи вриједност
```

Слика 3.7 - Примјер спречавања кориштења неприсутне вриједности као присутне

Типичан примјер је приступање непостојећем елементу вектора. Вектор *numbers* је иницијализован са вриједностима типа *i32* тако да приликом приступа неком његовом елементу методом *get* враћа вриједност *Option<i32>* типа. Ако је вриједност присутна, повратна вриједност ће бити варијанте *Some*(*i32* вриједност) и подудараће се са *Some(val) pattern*-ом који омогућава екстрактовање *i32* вриједности из *enum*-а и њено даље кориштење. Док у случају непостојања вриједности, *enum* варијабла ће бити варијанте *None* тако да вриједност неће ни бити доступна за кориштење. Свака грана *match* конструкције мора да евалуира у вриједност те стога мора да се заврши са изразом који се онда додјељује варијабли *value*. Због тога што *match* конструкција мора да обухвати све могуће вриједности које њен аргумент може имати, те због тога што свака грана мора да врати вриједност истог типа компајлер гарантује да ће варијабла *value* сигурно садржавати вриједност типа *i32* (прва грана враћа 4. елемент низа а друга експлицитно наведену вриједност 0). Ово значи да даљи код може користити вриједност варијабле *value* типа *i32* само ако је она присутна чиме се спречава велики број уобичајених багова [21].

3.3. Енкапсулација и систем модула

Rust омогућава сакривање имплементационих детаља кроз систем модула и приватности на нивоу ставки (типова, функција, подмодула, константи, *trait*-ова итд.). *Crate* је најмања јединица кода која може бити аргумент компајлирања и на основу ње се продукује библиотека или извршна датотека. *Crate* се састоји од модула који омогућавају груписање повезаних и сепарацију различитих функционалности у сврхе веће читљивости кода и лакше поновне употребе. Пакет представља више *crate*-ова који пружају скуп функционалности. Пакетима се управља *Cargo* пакет менаџером који је уједно и *build* систем. *Cargo* омогућава *build*, покретање и тестирање кода те многе друге функционалности као што су објава и инсталирање *crate*-ова са званичног *crate* регистра *crates*.io [21].

Модули омогућавају контролу приватности свих ставки унутар модула јер је код унутар њега подразумијевано приватан. Приватне ставке су детаљи интерне имплементације недоступне за вањску употребу. Ако је намјера експонирати одређени дио кода као јавни интерфејс и омогућити екстерном коду да зависи од њега потребно је модул и жељене ставке у њему учинити јавно доступним навођењем кључне ријечи *pub* испред њихових декларација. Осим што је могуће учинити структуре и енумерације јавним, такође је то могуће учинити и над њиховим елементима [21].

3.4. Управљање грешкама

У многим случајевима, Rust захтјева узимање у обзир могућности да ће се десити грешка током извршавања и преузимање одређених акција прије него што допусти компајлирање. Овај захтјев чини програме робуснијим јер омогућава детектовање и управљање грешкама прије него што оне заврше у продукцији. Rust користи тип *Result<T, E>* за грешке од којих се програм може опоравити и наставити извршавање, те *panic* макро који зауставља извршавање програма у случају критичне грешке као што је приступ елементу низа ван његових граница. Макрои су слични функцијама с тим да они враћају сам Rust код који се аутоматски уграђује у окружујући код прије компајлирања. Ово у великој мјери омогућава скраћивање *boilerplate* кода као што је то код за заустављање извршавања програма у случају *panic* макроа.

Како кориштење *Option<T> enum*-а рјешава проблем кориштења неприсутне вриједности као присутне, тако кориштење *Result<T, E> enum*-а рјешава проблем управљања грешкама од којих се програм може опоравити.

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Слика 3.8 - Result<T, E> enum

Ако операција враћа резултат типа *Result<T, E>* индицира се да она не мора бити извршена успјешно и да је након ње могуће наставити извршавање програма. Код који је захтијевао извршавање ове операције мора да обради резултат, најчешће користећи *match* конструкцију, да би могао користити евентуалну повратну вриједност која се налази у *Ok(T)* варијанти или да увиди грешку чије информације се налазе у *Err(E)* варијанти и одлучи о даљем извршавању [21].

3.5. Генерици

Rust подржава употребу генерика који омогућавају дефинисање потписа функција, метода и типова на такав начин да се они онда могу користити са различитим конкретним типовима. Кориштење генерика у дефиницијама омогућава изражавање њихових понашања и начина на које се они односе са другим генерицима и конкретним типовима без потребе да се зна шта ће се наћи на њиховом мјесту током компајлирања и извршавања. Приликом компајлирања Rust врши процес мономорфизације који подразумијева детектовање свих инстанци конкретног типа које користе генерички тип, те вршење замјене генеричке дефиниције са конкретном дефиницијом за сваки кориштен конкретни тип. Ово ефективно значи да се овако генерисане дефиниције не разликују од оних које су ручно написане, што нема ефекат на *runtime* перформансе. Кориштење генерика заједно са *traits* може бити веома експресивно и ефикасно по питању писања кода [21].

3.6. Traits и trait bound

Trait дефинише функционалност коју одређени тип може имати и дијелити са другим типовима. Сличан је интерфејсима из других програмских језика. Типови дијеле исто понашање ако је могуће позвати исте методе над њима. *Trait* групише потписе метода да би дефинисао скуп понашања потребних за неке сврхе. Ако тип треба да има понашање неког *trait*-а потребно је да имплементира тај *trait*. У комбинацији са генерицима омогућава ограничавање генеричких типова тако да они могу бити замијењени током компајлирања само са оним типовима који имају одређено *trait* понашање (*trait bounded* типови). Код који користи *trait bounded* тип има гаранцију која сва понашања тај тип испољава. Ако није потребно или није могуће знати конкретан тип који ће се наћи на мјесту генеричког типа током компајлирања потребно је користити *trait* објекат.

Овај објекат може бити замијењен са више различитих конкретних типова током извршавања те тако омогућава такозвани *dynamic dispatch*. Један од начина креирања *trait* објекта јесте употребом *Box<T>* типа који смјешта тип *T* на *heap*. Компајлер гарантује да сваки *trait* објекат имплементира спецификовани *trait* што спречава *runtime* грешку позивања метода над типовима који не имплементирају *trait* методе. Ово омогућава постизање неких од погодности као што су смјештање више различитих типова у једну колекцију или враћање више различитих типова у тијелу функције.

3.7. Конкурентност

Сигурно и ефикасно управљање конкурентним програмирањем је један од главних циљева Rust програмског језика. Кориштење *ownership* система и строге провјере типова, многе грешке конкурентности су грешке током компајлирања а не током извршавања. Rust омогућава кориштење нити које су имплементиране према 1:1 моделу гдје програм користи по једну OS нит за сваку Rust нит. Осим тога, Rust посједује и другачији приступ конкурентном програмирању од нити који се назива *async* систем. Оба ова приступа могу да користе конкурентност базирану на размјени порука и ону базирану на дијељењу података [21].

За кориштење размјене порука стандардна библиотека посједује имплементацију канала гдје се сваки канал састоји из предајника и пријемника. Канали најчешће користе *ownership* правило гдје је дозвољено да постоји само један власник због тога што након слања неке вриједности низ канал, код који је послао вриједност не би требало више да користи ту вриједност. С друге стране, конкурентност базирана на дијељењу података може захтијевати више власника неке вриједности из разлога што се очекује да више нити мијења и чита ту вриједност. Да би се остварило дијељење података Rust користи систем типова и специјалне примитиве конкурентности као што су *mutex*-и и *reference-counting* показивачи [21].

Mutex<T> тип пружа *lock* методу која враћа *MutexGuard* варијаблу која држи *lock* над *mutex*-ом све док је она у опсегу. *Lock* бива аутоматски отпуштен када *MutexGuard* варијабла изађе из опсега. Овиме се кроз систем типова осигурава преузимање *lock*-а и спречава заборављање његовог отпуштања. Због тога што је *mutex* потребно дијелити између више нити, потребно је да постоји више власника над њим. У ове сврхе се користи атомични *reference-counting* показивач *Arc<T>* који представља структуру која пружа методе за сигурну употребу вриједности неког типа од стране више власника у конкурентном контексту. Независно од овога, Rust и даље не може да спријечи неке од багова конкурентности као што су *deadlock* и *livelock* [21].

3.7.1. Async систем

Async систем омогућава грануларнију контролу над конкурентним извршавањем и мањи *overhead* у односу на нити. Састоји се из *future* концепта те *async* и *await* кључних ријечи који чине основу асинхроног програмирања у Rust-у. Сваки асинхрони код захтјева подешавање *async runtime*-а који управља детаљима извршавања асинхроног кода. Јединице извршавања *async runtime*-а су *futures*. *Future* је вриједност која не мора одмах бити доступна и која ће постати доступна у некој будућности. Навођењем *async* кључне ријечи испред блокова и функција специфицира се да они могу бити прекинути и поновно покренути од стране *runtime*-а. У позадини, овакав код ће бити трансформисан у *future*. Унутар *future*-а могуће је користити *await* кључну ријеч да се означи мјесто у коду гдје се може паузирати а затим и наставити његово извршавање. За вријеме паузе, контрола се препушта *async runtime*-у који може искористити ово вријеме за извршавање других *futures*-а док све вријеме провјерава да

ли је вриједност првобитног *future*-а доступна. Такође, могуће је и потпуно синхрони код учинити асинхроним експлицитним препуштањем контроле *runtime*-у специјалном функцијом *yield_now*. Ова функција је дио популарне *Tokio* библиотеке која, уосталом, садржи и *tokio runtime* који је најчешће коришћен *async runtime* у Rust-у, изразито када су у питању веб апликације [21].

3.8. Предности и недостаци програмског језика Rust

Концепти као што су систем типова, *ownership* и посуђивање гарантују сигурност меморије током компајлирања. Rust многе багове детектује за вријеме компајлирања и јасно указује на њихову локацију. Уз то, пружа већу контролу над меморијом без потребе за мануелним деалоцирањем меморије.

Непостојање *garbage collector*-а и многе *zero-cost* апстракције омогућавају веома високе *runtime* перформансе.

Rust подржава примјену многих концепата из функционалног програмирања као што су итератори, анонимне *inline* функције (*closures*), стримови итд. Дјелимично се сматра објектно-оријентисаним језиком јер подржава принцип објеката у виду структура и енумерација, те принцип енкапсулације кроз опцију кориштења кључне ријечи *pub* за различите дијелове кода. Принцип насљеђивања није у потпуности испуњен јер није могуће на тривијалан начин наслиједити поља из супер-структуре. Ипак, могуће је користити концепт полиморфизма употребом структура и *trait* објеката.

Због тога што је Rust програмски језик намијењен за системско програмирање, могуће је одабрати ниво апстракције и по потреби спустити се на ниже нивое него што је то могуће или практично у другим језицима. Rust омогућава коришћење *unsafe* блокова који дозвољавају заобилажење неких сигурносних провјера *ownership* система, али уз преузимање одговорности од стране програмера. Такође, довољно је експресиван и прилагођен са писање кода на високом нивоу апстракције који се компајлира у програме са перформансама *low-level* кода.

Због јединственог *ownership* система може бити тежак за учење у почетку. Такође сва додатна сигурност меморије коју гарантује негативно се одражава на вријеме компајлирања [21].

4. Развој микросервиса у програмском језику Rust

За имплементацију микросервисног система у програмском језику Rust најчешће је неопходно обезбједити сљедећу логику:

- Сервера и клијента HTTP-а
- Серијализацију и десеријализацију података
- Клијента релационе базе података и ORM
- Клијента система за размјену порука
- Аутентификацију и ауторизацију
- Опсервабилност
- Конфигурабилност

4.1. Сервер и клијент HTTP-а

За API опште намјене као што је REST API популарно је користити HTTP сервер у оквиру веб апликационог развојног оквира као што је *Actix Web* или *Axum*.

Actix Web је практичан, брз и релативно моћан *micro-framework*. Довољно је моћан да подржи HTTP/1, HTTP/2 и TLS без *boilerplate* кода те је као такав идеалан за брз развој микросервиса. Осим HTTP сервера пружа и функционалности рутирања, *middleware*-а, предпроцесирања захтјева, постпроцесирања одговора итд. Уз то, омогућава и декларативно строго типизирано парсирање захтјева кориштењем екстрактора у виду *trait*-а. Постоје разне *out-of-the-box* имплементације екстрактора као што су *Path, Query, Json* итд. које заједно са библиотекама за десеријализацију омогућавају аутоматску екстракцију овако форматираних података у Rust типове. Такође је олакшано и креирање тијела одговора на основу Rust типова у разним форматима (JSON, *plain text,* датотеке, бинарни подаци итд.). За дијељење стања између *request handler* функција (нпр. конфигурације или *pool* конекција ка бази података) користи *dependency injection* механизам што елиминише потребу за мануелним прослеђивањем стања из дефиниције сервера до одговарајућих *request handler* функција. Грешке *request handler* функција су унифициране у један *trait* чијом имплементацијом је олакшано управљање грешкама сервера.

Axum је веб развојни оквир фокусиран на лакоћу употребе и модуларност. Такође пружа функционалности управљања и рутирања захтјева с тим да се за *middleware*, предпроцесирање захтјева и постпроцесирање одговора ослања на богату *tower* библиотеку. За употребу TLS протокола модуларан је са *rustls* и *openssl* библиотеком. Његов API има функционални стил за разлику од *Actix Web-а* који у великој мјери користи макрое који су генерално тежи за дебаговање. Такође користи

dependency injection али употребом једног глобалног екстрактора стања с тим да омогућава експлицитно дефинисање грануларније екстракције. Као и *Actix Web* користи темељно тестирани *runtime* из *tokio* библиотеке.

Tower библиотека садржи велики број модуларних и поновно употребљивих *middleware* компоненти за писање робусних клијената и сервера разних протокола. Веома је корисна за обезбјеђивање функционалности микросервиса као што су *load-balancing*, TLS, аутентификација, ауторизација, *timeouts*, *retries*, *rate-limiting*, *circuit breaking*, *tracing*, логовање итд.

За имплементацију HTTP клијента на високом нивоу апстракције може се искористити *reqwest* библиотека. Пружа функционалности као што асинхрони и блокирајући клијент, разни типови тијела захтјева (*plain text*, JSON, *url-encoded*), TLS, колачићи итд. Ако је потребан већи ниво контроле може се користити *hyper* библиотека која је на нижем нивоу апстракције од *reqwest* библиотеке. Она је веома често основни градивни блок других библиотека и развојних оквира као што су *Axum* и сам *reqwest*.

За имплементацију gRPC сервера и клијента може се искористити *tonic* библиотека која је фокусирана на перформансе, интероперабилност и флексибилност. Представља генеричку имплементацију gRPC-а путем HTTP/2 протокола и омогућава употребу различитих gRPC формата за енкодовање од којих је најзаступљенији *Protobuf*. *Schema-first* приступ је подржан уз помоћ *codegen* функционалности која на основу уговорних *.proto* датотека аутоматски генерише сав неопходан *low-level* код везан за мрежне операције, HTTP/2 *framing,* бинарно енкодовање итд. Ово омогућава кориснику ове библиотеке да је користи са високог нивоа апстракције фокусирајући се само на дефиниције и позиве удаљених процедура, њихове аргументе и одговоре. *Tonic* је модуларан са *tower* библиотеком у сврхе кориштења *middleware-а* те са *rustls* библиотеком у сврхе кориштења TLS-а.

За имплементацију асинхроног GraphQL сервера може се искористити *async-graphql* библиотека која подржава *code-first* приступ дефинисања шеме API-ја. На основу строго дефинисаних Rust типова и једноставних макроа ова библиотека креира шему на основу које валидира долазеће упите и врши интроспекцију. За GraphQL независне функционалности као што су HTTP сервер, рутирање захтјева и враћање одговора потребно је и лако извршити интеграцију са неким од веб развојних оквира као што су *Actix Web* и *Axum*.

За имплементацију асинхроног GraphQL клијента може се искористити *graphql-client* библиотека која подржава *schema-first* приступ. Једноставним навођењем макроа испред структуре која представља захтјев и на основу шеме и *query* датотека генерише код за слање захтјева и парсирање одговора. На тај начин, употребом Rust система типова валидира шему током компајлирања што спречава *runtime* багове слања неисправних упита.

4.2. Серијализација и десеријализација података

Serde је развојни оквир за серијализацију и десеријализацију Rust структура података на ефикасан и генеричан начин. Изграђен је на Rust *trait* систему који му омогућава да уведе додатни слој који прекида спрегу између Rust типова и разних формата података (JSON, YAML, *MessagePack* итд.). Након што тип имплементира одговарајуће *serde trait*-ове он постаје спреман за серијализацију и десеријализацију из било ког формата и у било који формат. Велики број библиотека и развојних оквира као што су *Actix Web, Axum, reqwest* се ослања на *Serde* библиотеку.

За серијализацију и десеријализацију података у *Protobuf* формат може се искористити *prost* библиотека. Ова библиотека на основу *.proto* датотека генерише Rust типове и сав неопходан код за њихову серијализацију и десеријализацију.

4.3. Клијент релационе базе података и ORM

За имплементацију SQL клијента са асинхроним интерфејсом може се искористити *sqlx* библиотека. Ова библиотека провјерава исправност SQL упита тако што се повезује са базом података током компајлирања што спречава *runtime* грешке слања неисправних упита. Ово може представљати проблем у неким CI/CD окружењима ако не постоји активна база података па је стога могуће креирати *.sqlx cache* података и користити га током компајлирања без потребе за постојањем активне базе података. За писање упита не пружа DSL (*Domain Specific Language*) већ захтијева директно кориштење SQL-а. Не представља ORM већ *low-level* SQL скуп алата па је потребно мануелно дефинисати моделе података и њихова спајања. Мапирање резултата у Rust типове је аутоматизовано уз помоћ одређених макроа који током компајлирања могу да провјере да ли Rust тип може представити податке из одговора SQL упита. Подржава трансакције и додатно, ова библиотека има уграђен *connection pooling* механизам и пружа могућност параметризованих упита за заштиту од SQL *Injection* напада.

За имплементацију SQL клијента без директног кориштења SQL-а за писање упита може се употријебити *diesel* библиотека. Такође као и *sqlx* провјерава сигурност упита током компајлирања али на начин да уз помоћ CLI алата аутоматски генерише Rust код који репрезентује мапирану цјелокупну шему базе података. Представља потпуно функционалан ORM гдје су све табеле представљене типовима а операције као што су филтрирање, сортирање и спајање се врше позивима метода. Сам API дизајн не омогућава SQL *Injection* јер се сви позиви API-ја транслирају у параметризоване упите с тим да постоји и могућност директног писања SQL упита. Клијент је синхроне природе тако да, ако се користи *async runtime* у оквиру веб сервера упите треба извршавати на засебним нитима предвиђеним за блокирајуће операције да не би дошло до блокирања нити веб сервера. Подржава трансакције а за *connection pooling* се ослања на друге библиотеке. Постоји новија *diesel-async* библиотека која је екстензија овој библиотеци и пружа асинхрони клијент и уграђени *connection pooling* механизам.

4.4. Клијент система за размјену порука

За имплементацију асинхроног AMQP клијента у верзији 0.9.1 коју користи *RabbitMQ* може се употријебити *lapin* библиотека. Користи *tokio runtime* и пружа API базиран на *async-await* механизму. Подржава све захтјеве из спецификације као што су канали, редови, *exchanges*, објављивање и конзумирање порука те слање потврда о успјешности објављивања и процесирања поруке. Лако се интегрише се TLS библиотекама као што су *native-tls* и *rustls*. Да би се повећале перформансе микросервиса може се искористити *connection pool* библиотека *deadpool-lapin* која пружа логику за креирање и рециклирање AMQP конекција, аутоматску реконекцију итд.

4.5. Аутентификација, ауторизација и остали сигурносни аспекти

Стандардна Rust библиотека за рад са JSON веб токенима је *jsonwebtoken*. Она омогућава креирање, валидацију и декодовање JSON веб токена кориштењем разних криптографских алгоритама.

За хеширање лозинки може се користити *argon2* или *bcrypt* библиотека. *Argon2* библиотека је сигурнија и има већи ниво конфигурабилности али је спорија и тежа по питању меморије.

Casbin је библиотека за ауторизацију која подржава разне моделе контроле приступа као што су листе (ACL), оне базиране на улогама (RBAC) и атрибутима (ABAC) итд. Омогућава висок ниво флексибилности јер се модел дефинише у конфигурационој датотеци а конкретне политике приступа се могу чувати у разним складиштима података кориштењем адаптера као што су *sqlx-adapter*, *diesel-adapter*, *filesystem-adapter* итд. Ова библиотека је веома погодна за спровођење контроле приступа микросервису како од стране корисника тако и од стране других микросервиса. Такође, олакшана је контрола приступа REST API-ју гдје се ресурсу приступа путем URI-ја и HTTP методе јер је могуће аутоматски провјеравати HTTP методе, *wildcard* знакове и параметре у путањама (нпр. /*resource*/*, /resource/:*id*).

4.6. Опсервабилност

Библиотека *log* представља стандардну Rust библиотеку за пружање општег API-ја за логовање. Постоји велики број имплементација ове библиотеке од којих је најпознатија *env_logger* која омогућава подешавање логовања путем варијабли окружења и филтрирање логова на стандардни излаз.

Генерално, логови имају низак ниво структурисаности и тешко је извршити њихову корелацију унутар једног пословног задатка. За рјешавање ових проблема препоручено је користити *tracing* библиотеку која представља *tracing* развојни оквир који омогућава праћење извршавања логичког тока пословних задатка и прикупљање

њихових структурисаних догађај-базираних дијагностичких информација у асинхроном и дистрибуираном окружењу. Користи *span* концепт са контекстним подацима да представи и опише период извршавања посла. *Span* може да садржи више догађаја у времену са додатним структурисаним подацима. Заједно са другим компатибилним датотекама омогућава слање ових *observability* података на дистрибуиране *tracing backend* системе који имплементирају OpenTelemetry стандард.

4.7. Конфигурабилност

Библиотека *config* је стандардна библиотека за управљање конфигурацијом апликације. Омогућава подразумијевану конфигурацију те учитавање конфигурације из варијабли окружења и различитих формата фајлова (TOML, YAML, JSON) и све оне могу бити хијерархијски комбиноване у циљу прилагођавања понашања апликације у различитим *deployment* окружењима. За десеријализацију конфигурације у строго типизиране структуре користи *serde* библиотеку.

5. Практични рад

У практичном дијелу рада је реализована *Weather Aggregator* апликација са микросервисном архитектуром кориштењем Rust програмског језика. Објашњено је како је Rust помогао развоју микросервиса и детаљно је демонстрирана употреба адекватних Rust библиотека и развојних оквира.

5.1. Опис апликације

Weather Aggregator је апликација која агрегира податке о тренутним временским условима од више јавно доступних провајдера. Смањује утицај потенцијалних отказа провајдера јер наставља да враћа најновије резултате док год је бар један провајдер доступан. Штавише, може некада да врати резултате чак и када ниједан провајдер није доступан јер чува привремене податке у кешу што уједно смањује латенцију и број API позива. Додатно, апликација пружа и могућност регистрације корисника и чување њихових омиљених и историјских локација као и одређених подешавања.

5.2. Спецификација и архитектура апликације

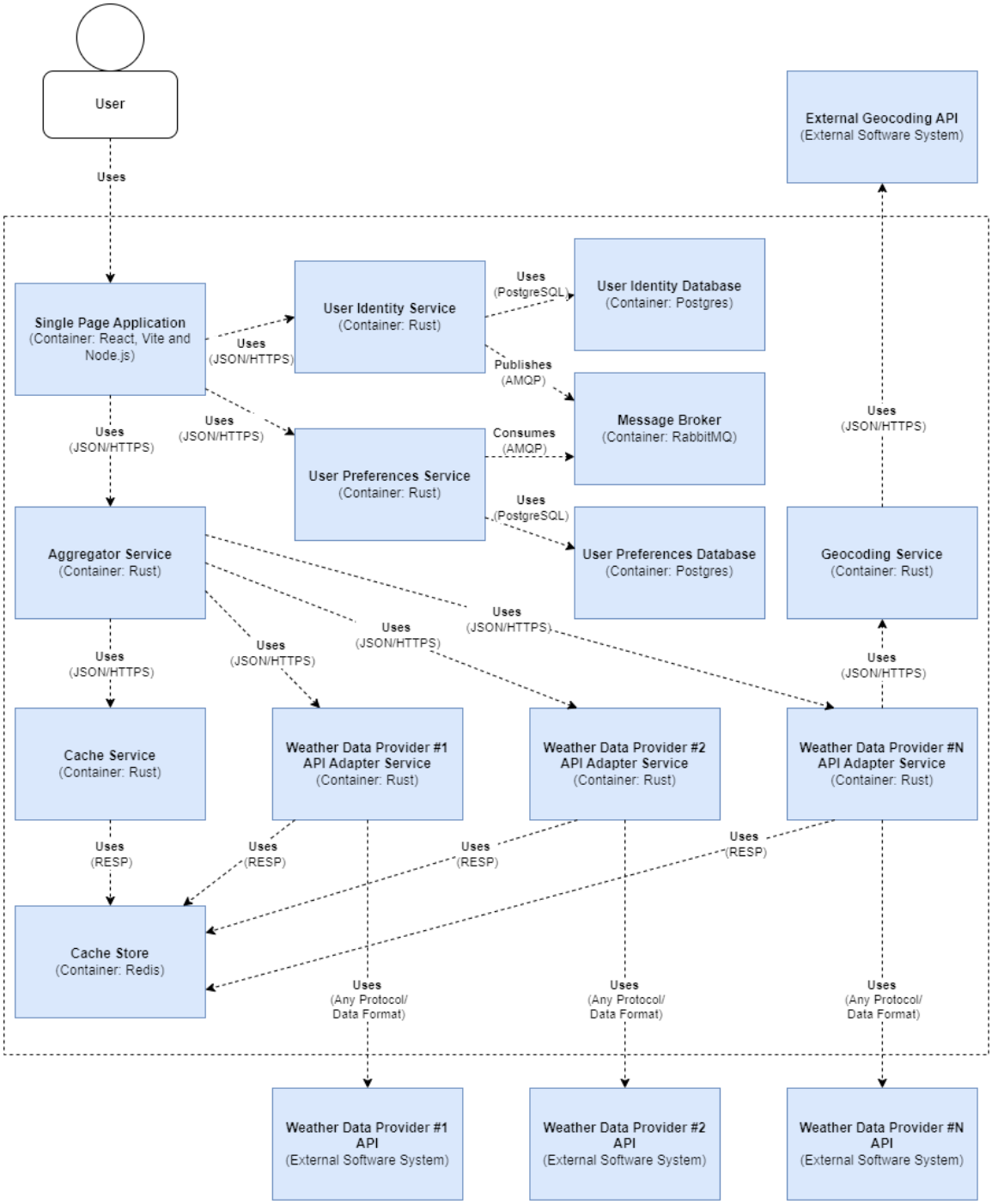
Цјелокупан систем се састоји од клијентске и серверске апликације, додатних инфраструктурних компонената и екстерних API-ја. Конкретне компоненте су (слика 5.1):

1. Више екстерних API-ја као провајдери података о временским условима. Сваки од њих има одређен *rate limit* и специфичан интерфејс.
2. Више API адаптер микросервиса који прилагођавају временске резултате више провајдера за *upstream* сервисе. Сваки адаптер сервис врши специфичан посао везан за једног провајдера као што су позив API-ја, *rate limiting* и трансформација резултата у униформан формат.
3. Микросервис за геокодирање који претвара називе локација чији су временски услови затражени у географске координате позивајући екстерни API за геокодирање. Овај сервис је потребан неким адаптер сервисима јер неки екстерни API-ји захтијевају географске координате као параметре за упите о времену.
4. Агрегатор микросервис који позива адаптер сервисе и агрегира њихове резултате на основу сљедећих стратегија:
    a. *Averaging* – Израчунава средњу вриједност свих заједничких параметара временских података више провајдера (нпр. тренутна температура или смјер вјетра).

b. *Priority* – Враћа резултате по конфигурисаном приоритету провајдера.

c. *Fallback* – Враћа резултате произвољног провајдера који је одговорио.

Додатно, овај сервис кешира податке о времену и користи их позивајући *Cache* сервис.



Слика 5.1 Дијаграм контејнера

5. *Cache servis* који чува привремене податке о времену у Redis складишту података. На основу имена локације или координата враћа кеширане податке.

6. *Redis* складиште података – Чува податке о времену у формату *кључ:вриједност* гдје је кључ у формату *држава:ентитет:локација*. Додатно, кешира податке и по *geohash* кључу који представља стринг репрезентацију одређеног распона координата који припада једном сегменту са одређеном величином на мапи (нпр. дужина *geohash* стринга 5 одговара величини сегмента од 4.9km * 4.9km). Ово значи да, ако се двије локације налазе унутар истог сегмента, неће бити потребно кешirати податке за обје локације већ само за прву а приликом тражења друге десиће се *cache hit*. Ово знатно смањује број API позива. Такође, *Redis store* користе и сви адаптер сервиси као и сервис за геокодирање да би чували податке о томе колико је упућено позива ка екстерним API сервисима у посљедњих неколико минута. Кориштење *Redis store-а* за чување ових ефемерних података постиже то да сервиси остану без стања и тривијални за скалирање.

7. *User Identity* микросервис који омогућава регистрацију, пријаву, одјаву, освјежавање JWT-а те добављање информација о кориснику. Податке о корисницима чува у *Postgres* бази података која је само њему доступна. Омогућава пријаву како стандардних тако и анонимних корисника. Корисници су пријављени путем JWT-а који се налази у њиховом дугоживућем колачићу тако да обје врсте корисника имају слично корисничко искуство. Анонимни корисник се у сваком тренутку може регистровати и одабрати да сачува претходно запамћене податке уз помоћ његовог колачића.

8. *User Preferences* микросервис који чува корисникове претходно претраживане локације, његову омиљену локацију и систем мјерења временских података. Такође користи само своју *Postgres* базу података.

9. *RabbitMQ Message Broker* путем којег посредују *User Identity* и *User Preferences* сервиси да би одржали конзистентност података између њихових база података. За ово користе архитектуру вођену догађајима. Нпр. *Identity* сервис објављује догађај да је анонимни корисник регистрован а *Preferences* сервис га конзумира и усклађује своје стање о кориснику у бази података.

10. Клијентска веб апликација развијена кориштењем *Vite* алата, *React* библиотеке и *Node.js* сервера.

Сви микросервиси излажу API у REST стилу с циљем одржавања могућности тривијалног скалирања. Сви привремени подаци се чувају у *Redis* складишту података. Комуникација између микросервиса се одвија кореографијом без централног чвора. Успостављена је конфигурабилна mTLS комуникација тако да сви сервиси који не

треба да буду јавно доступни прихватају конекције само од оних сервиса који имају адекватан дигитални сертификат. Агрегатор, *User Identity* и *User Preferences* сервиси су јавно доступни.

Комуникација са додатним инфраструктурним компонентама као што су базе података и брокер порука се одвија путем конфигурабилног mTLS-а. Такође, подешена је и аутентификација на апликативном нивоу. Сва ова подешавања је могуће измијенити у конфигурационим датотекама у оквиру *Docker*-а.

Архитектура појединачних микросервиса је дизајнирана по *Controller-Service-Repository* шаблону. Контролери су задужени за управљање захтјевима и одговорима, валидацију улаза и позивање сервисног слоја. Сервисни слој садржи сву пословну логику и позива слој репозиторијума који апстрахује приступ перзистентном слоју. Сама доменска логика у сервисном слоју је организована као трансакциони скрипт јер су пословне функционалности релативно једноставне и прихватљиво је да се извршавају процедурално.

5.3. Процес развоја

За имплементацију микросервиса кориштен је API-вођен дизајн који подразумијева писање кода почевши од његовог API-ја. Интерфејс се структурише на начин на који га клијент жели позвати. Након тога се имплементирају позиване функционалности у складу са дефиницијом интерфејса. Овај приступ је кориштен између било која 2 слоја у апликацији (нпр. између клијента и API-ја или између контролера и сервиса). API-вођен дизајн је комбинован са компајлер-вођеним дизајном гдје се након написаног интерфејса провјеравају грешке у коду специјалним *cargo check* алатом. Овај алат не врши генерисање бинарног кода и линковање већ само прву фазу компајлирања која подразумијева парсирање кода, провјеру типова, *ownership* правила, синтаксних грешака итд. Након ових провјера уједно се и опширно указује на неимплементиране функционалности које се затим имплементирају и тако се овај процес развоја наставља све до најнижих слојева.

У наставку су детаљно описане кориштене библиотеке и развојни оквири уз образложење њиховог кориштења.

5.3.1. Веб развојни оквир

Као веб развојни оквир кориштен је *Actix Web* због своје дугогодишње стабилности и богатог екосистема. Његова основа је *HttpServer* структура. Она управља свим транспортним задацима као што су: одређивање на којем сокету апликација слуша долазеће захтјеве, ограничавање броја конекција по јединици времена, подешавање TLS-а итд. За инстанцирање HTTP сервера и његово подешавање користи се *new* функција и *builder* шаблон. Аргумент ове методе је *lambda* функција која враћа инстанцу *App* структуре. Ова структура садржи сву апликациону логику као што су рутирање захтјева, *middleware-u, request handler-u* итд. Примјер креирања и покретања HTTP сервера са апликацијом у сервису за геокодирање дат је на сљедећој слици:

```
pub fn run(tcp_listener: TcpListener) -> std::io::Result<Server> {
    let mut server = HttpServer::new(move || {
        App::new()
            .app_data(http_client.clone())
            .app_data(geocoding_service.clone())
            .app_data(configuration_settings.clone())
            .app_data(redis_pool.clone())
            .service(
                web::scope("/api/v1")
                    .configure(geocoding_controller::routes)
            )
            .route("/health_check", web::get().to(health_check))
    }).listen(tcp_listener)?;
    return Ok(server.run());
}
```

Слика 5.2 Функција за креирање и покретање HTTP сервера

За везивање за сокет искориштена је *listen* метода. Овој методи је прослијеђена инстанца *TcpListener* типа која се креира на основу IP адресе и порта. На крају примјера покреће се сервер и враћа као *future* позивајућој функцији која је у овом случају *main*. Да би се омогућило кориштење *await* кључне ријечи над *future*-ом у *main* функцији, потребно је да она буде декларисана као *async* функција и да буде декорисана *actix_web::main* макроом (слика 5.3). Овај макро ће током компајлирања извршити трансформацију асинхроне *main* функције у синхрону и уградити сав неопходан *boilerplate* код. Тај код је креирање и стартовање *tokio async runtime*-а којем се прослеђује покренути сервер као *future*. Затим *async runtime* неограничено дуго блокира главну нит да програм не би прекинуо са извршавањем.

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    let res = run(
        listener,
    )?.await;
    return res;
}
```

Слика 5.3 Декорисана *async main* функција

App инстанца у примјеру 5.2 се такође креира по *builder* шаблону и то на начин да јој се путем методе *app_data()* прослеђују подаци стања који се дијеле између више *handler*-а. Због тога што *HttpServer* може инстанцирати *App* структуру и сва њена стања за сваку своју нит посебно потребно је стања омотати унутар *web::Data<T>* структуре. Ова структура интерно користи *Arc<T>* тип тако да, ако се позове *clone()* метода над њом као у примјеру, свака нова *App* инстанца има клонирану *Arc<T>* инстанцу. Ово значи да свака нит заправо ради са истом инстанцом стања *T*. Неки од тих стања су *redis_pool* и *geocoding_service* који се користе од стране контролера који је представљен *handler*-ом. Ако је потребно мијењати стање од стране више нити потребно га је омотати унутар *Mutex<T>* или *RwLock<T>* типа те користити њихове одговарајуће методе.

Actix Web омогућава флексибилно додавање *endpoint*-а у апликацију на сљедеће начине:

1. Централизовано позивањем *route* методе над *App* инстанцом. Ова метода прима путању и *Route* инстанцу. *Route* структура комбинује *handler*-е са скупом типова који имплементирају *Guard trait.* Један од тих типова је *guard::Get()* који се користи од стране *web::get()* методе у примјеру. Рута у примјеру одговара захтјеву само ако је захтјев упућен HTTP GET методом на */health_check* путању након чега се позива *health_check request handler.*

2. Децентрализовано користећи макрое или груписањем рута. Кориштење макроа је најједноставнији начин у којем се сваки *handler* декорише макроом који специфицира путању и HTTP методу (нпр. *#[get("/")]* или *#[post("/echo")]*). Регистровање овако декорисаних *handler*-а се постиже уланчавањем позива *service* методе над *App* инстанцом (нпр. *.service(handler1_name).service(handler2_name)*). У контексту *service* методе *handler*-и и њихове путање су ресурси и могуће је регистровати више ресурса одједном јер *service* метода као аргумент прихвата и *Scope* инстанцу. Ова структура представља скуп ресурса са заједничким префиксом путање (у примјеру *"/api/v1"*). Она уједно омогућава екстернализацију дефинисања рута кориштењем *configure* методе над *Scope* инстанцом која прихвата *ServiceConfig* тип (слике 5.2 и 5.4). Овај начин постиже поновну употребљивост јер се руте могу груписати у разним дијеловима кода као што је други модул.

```
pub fn routes(cfg: &mut web::ServiceConfig) {
    cfg.route("/weather_by_coordinates", web::get().to(weather_by_coordinates))
       .route("/weather_by_location", web::get().to(weather_by_location))
       .route("/weather_by_ip_address", web::get().to(weather_by_ip_address));
}
```

Слика 5.4 Груписање рута уз помоћ *ServiceConfig* типа

Сами *request handler-u* су асинхроне функције које могу имати до 12 параметара а над њима се врши *dependency injection* механизам. Ови параметри могу бити подаци стања који су најчешће омотани у *web::Data<T>* тип и било који тип који имплементира *FromRequest trait.* Тип који имплементира овај *trait* може бити екстрактован из тијела или заглавља HTTP захтјева. Неки од тих типова су *Path, Json* и *Query*.

```
async fn weather_by_coordinates(
    query: Query<WeatherByCoordinatesRequest>,
    http_client: web::Data<ClientWithMiddleware>,
    /* ... */
) -> Result<impl Responder, GenericServiceError> {
    return Ok(Json(weather_service.get_weather(query.as_ref(), /* ... */).await?));
}
```

Слика 5.5 Примјер *request handler*-а

На слици 5.5 је дат примјер *request handler*-а са екстракцијом података стања и *query string*-а захтјева ка агрегатор сервису. Параметар генеричког типа *query* је *WeatherByCoordinatesRequest* структура која представља Rust репрезентацију скупа *query* параметара. Она мора да имплементира *serde::Deserialize trait* да би *handler* могао исправно да десеријализује податке у њу из *query string*-а.

Приликом успјешне обраде захтјева *request handler* враћа тип који имплементира *Responder trait*. Ови типови знају како да креирају HTTP одговор. Неки од тих типова су *Json<T>* и *Html* који аутоматски постављају тијело захтјева са JSON или HTML подацима и одговарајуће *content-type* заглавље те враћају одговор са статусним кодом 200. Тип унутар *Json<T>* типа мора да имплементира *serde::Serialize trait* као што је то у примјеру повратна вриједност *get_weather* функције. Такође, постоји и *HttpResponse* тип који омогућава мануелно постављање статусног кода, тијела и заглавља одговора.

Приликом неуспјешне обраде захтјева *handler*-и враћају тип који имплементира *ResponseError trait*. Један овакав тип је *Error* тип из стандардне библиотеке који враћа статусни код 500 и релативно дескриптиван текст грешке. Због тога је у овом пројекту кориштен *GenericServiceError* тип који мапира све грешке које се могу десити у једном сервису у униформан формат (слика 5.6).

```
#[derive(Serialize)]
pub struct GenericServiceError {
    pub error: GenericServiceErrorDetails,
}

#[derive(Serialize)]
pub struct GenericServiceErrorDetails {
    pub code: ServiceError,
    pub code_numeric: u16,
    pub message: String,
    pub timestamp: DateTime<Utc>,
}
```

Слика 5.6 Генерички тип грешке сервиса

```
impl ResponseError for GenericServiceError {
    fn status_code(&self) -> StatusCode {
        return match self.error.code {
            ServiceError::RequestValidationError(_)=> StatusCode::BAD_REQUEST,
            ServiceError::ExpiredRefreshTokenError(_) => StatusCode::FORBIDDEN,
            _ => StatusCode::INTERNAL_SERVER_ERROR,
        };
    }
    fn error_response(&self) -> HttpResponse {
            let err = self.sanitize_message();
            return HttpResponse::build(self_status_code()).json(err);
        }
    }
}
```

Слика 5.7 Имплементација *ResponseError trait*-а од стране *GenericServiceError* типа

На слици 5.7 је дат начин на који овај тип имплементира *ResponseError trait.* Унутар *status_code* методе свака сервис-специфична грешка која се може десити бива мапирана у одговарајући HTTP код грешке. Унутар *error_response* методе врши се санитизација одређених грешака позивом *sanitize_message* методе а затим се уз помоћ *HttpResponse* типа конструише HTTP одговор грешке у виду JSON-а.

5.3.2. Клијент HTTP протокола

За имплементацију клијента HTTP-а искориштена је *reqwest* библиотека. За креирање асинхроног клијента у виду *Client* инстанце кориштена је *ClientBuilder* структура која омогућава подешавање разних особина клијента као што су максимални број редирекција, *timeout,* TLS итд. *Client* инстанца подразумијевано интерно користи *pool* конекција. Такође, она интерно користи и *Arc<T>* тип тако да је сигурно извршити њено клонирање и кориштење од стране више нити веб сервера.

За креирање захтјева користи се *builder* шаблон уз помоћ *RequestBuilder* инстанце. Ову инстанцу враћају Rust мапиране HTTP методе *Client* инстанце као што су *post(), get(), put()* итд. Ове методе као аргумент прихватају тип који имплементира *IntoUrl trait* од којих је један и стринг који они накнадно енкодују у URL. Подаци се могу слати као *query* параметри методом *query* која као аргумент прихвата тип који имплементира *serde::Serialize trait*. Неки од ових типова су стандардни типови као што је *HashMap* и *slice* парова. Као тијело захтјева аутоматски се могу послати JSON форматирани подаци *json()* методом која такође прихвата тип који имплементира *serde::Serialize trait.* Примјер креирања и слања захтјева дат је на сљедећој слици:

```
let response = client
    .get(format!("{}/{}", base_api_url, current_weather_endpoint))
    .query(&[
        ("q", q_argument.as_str()),
        ("api_key", provider_settings.api_key.expose_secret().as_str()),
      ])
    .send()
    .await
    .map_err(|e| {
        AdapterServiceError::ConnectionError(Some(e.to_string()))
    })?;
```

Слика 5.8 Примјер креирања и слања *reqwest* захтјева

Query параметри се креирају у виду *slice*-а парова као *&[(q, arg1), (key, arg2)]*. Елемент који је упарен са *"api_key"* стрингом је омотан у *SecretString<T>* тип из *secrecy* библиотеке да би се спријечио његов приказ приликом логовања. Добављање његове вриједности се врши методом *expose_secret()* у примјеру. Слање креираног захтјева се врши *send()* методом након чега се асинхроно чека на одговор кључном ријечи *await*. У случају грешке она се мапира у одговарајућу грешку сервиса и пропагира назад кроз *call stack* употребом ? оператора. JSON тијело из одговора у виду *Response* инстанце је могуће аутоматски десеријализовати у Rust тип позивом генеричке функције *json<T>()* ако тај тип имплементира *serde::Deserialize trait.*

5.3.3. Серијализација и десеријализација података

За серијализацију и десеријализацију података кориштена је *serde* библиотека која је дубоко интегрисана у друге развојне оквире и библиотеке те је стога, њено кориштење готово немогуће избјећи.

У примјеру 5.5, да би тип *WeatherByCoordinatesRequest* могао бити десеријализован из *query string*-а мора да имплементира *Deserialize trait*. Овај тип је омотан у *Query* екстрактор који врши парсирање *query string*-а у привремене типове а затим позива униформни интерфејс *Deserialize trait*-а да креира *WeatherByCoordinatesRequest* инстанцу. Ове акције аутоматски иницира *Actix Web* након рутирања захтјева ка *weather_by_coordinates handler*-у. У случају неуспјешне десеријализације враћа се дескриптивна грешка крајњем кориснику. У овом пројекту извршено је њено пресретање и враћање униформне грешке користећи *custom query_error_handler*-а који се конфигурише приликом покретања сервера.

Додатно, због тога што *WeatherByCoordinatesRequest* тип имплементира *Deserialize trait* он може бити десеријализован из било којег формата који адекватно користи овај *trait* (као што су *Json<T>* и *Path<T>* из *actix_web* библиотеке). Најчешћи начин имплементације *Deserialize trait*-а је навођењем *derive* макроа прије дефиниције одредишног типа (слика 5.9).

```
#[derive(Deserialize, Serialize)]
pub struct WeatherRequestByCoordinates {
    #[validate(range(min = -90.0, max = 90.0))]
    pub lat: f64,
    #[validate(range(min = -180.0, max = 180.0))]
    pub lon: f64,
}
```

Слика 5.9 Тип који имплементира *Deserialize* и *Serialize trait*

Ако се користи *Deserialize trait* аутоматски се врши провјера грешака као што су погрешни типови, недостајућа поља итд. За стриктније провјере улазних података као што су опсег вриједности, минималан и максималан број знакова стринга без кориштења мануелне десеријализације искориштена је *validator* библиотека. Она путем макроа *validate* пружа једноставан начин валидације поља структуре. Услов за валидацију може бити повратна вриједност уграђене функције као што је то у примјеру *range* функција или мануелно написане функције са одговарајућим потписом. Осим кориштења макроа потребно је и експлицитно позвати *validate* методу у *request handler*-у. Да би се ово избјегло, у пројекту је искориштена *actix-web-validator* библиотека која садржи екстракторе који омотавају *actix-web* екстракторе и аутоматски позивају *validate* методу из *validator* библиотеке. Ако је валидација неуспјешна, извршавање кода *request handler*-а неће бити започето и биће враћена грешка валидације.

Приликом инстанцирања типова који представљају доменске објекте искориштен је типски-вођен развој. Он подразумијева да се доменска ограничења спроводе строгом компајлерском провјером типова. Ако је доменски објекат са

доменским ограничењем (нпр. тип *UserEntity* садржи поље *email* чија вриједност је валидан *email*) доступан за употребу другим дијеловима кода, онда је он сигурно претходно валидиран. Ова валидација се постиже тако што се изворишни подаци доменског објекта парсирају у одговарајуће типове (слика 5.10).

```
impl TryFrom<&RegisterStandardUserRequest> for UserEntity {
    type Error = UserError;

    fn try_from(req: &RegisterStandardUserRequest) -> Result<Self, Self::Error> {
        let pwd_hash: UserPassword = UserPassword::parse(req.pwd)?;
        let email: UserEmail = UserEmail::parse(req.email)?;

        Ok(UserEntity {
            id: Uuid::new_v4(),
            email: Some(email),
            password_hash: Some(pwd_hash),
            user_type: UserType::STANDARD,
        })
}
```

Слика 5.10 Примјер креирања доменског објекта парсирањем захтјева

Изворишни подаци доменских објеката су поља *RegisterStandardUserRequest* захтјева који се парсирају у типове *UserPassword* и *UserEmail* позивом *parse* методе која уједно ради и доменску валидацију. На слици 5.11 је приказан примјер *UserEmail* структуре која се инстанцира *parse* функцијом након успјешне валидације *email* стринга. Ако је парсирање неуспјешно доменски објекат неће бити инстанциран.

```
#[derive(Debug, Clone, Serialize)]
pub struct UserEmail(pub String);

impl UserEmail {
    pub fn parse(s: &String) -> Result<UserEmail, UserError> {
        if validate_email(&s) return Ok(UserEmail(s.clone()));
        else return Err(UserError::EmailInIncorrectFormat);
    }
}
```

Слика 5.11 Примјер парсирања *email*-а из захтјева у *email* доменског објекта

Инстанце *UserPassword* и *UserEmail* типова су заправо само омотачи око инстанце *String* типа с тим да су, за разлику од инстанце *String* типа, прошли доменску валидацију и даљи код их може користити као такве без потребе за накнадном валидацијом.

5.3.4. SQL клијент

За имплементацију SQL клијента искориштена је *sqlx* библиотека јер пружа писање SQL упита директно без потребе за прилагођавањем новом DSL језику што одговара релативно једноставаном доменском моделу ове апликације.

За конфигурисање конекције ка PostgreSQL бази података искориштена је *PgConnectOptions* структура. Она омогућава једноставно подешавање конекције

парсирањем URL стринга или кориштењем *builder* шаблона. Приступ шаблона омогућава већи ниво конфигурабилности и то на начин да се уланчавају позиви метода као што су *host, username, log_statements, sslmode* итд. За конфигурисање *pool*-а конекција може се искористити *PgPoolOptions* структура. Она омогућава методе за подешавање максималног или минималног броја конекција у *pool*-у, њиховог максималног животног вијека итд. За креирање *PgPool* менаџера *pool* конекција повезивањем са базом података може се искористити *connect* или *connect_with* метода. *Connect* прима *Postgres* URL стринг док *connect_with* прима инстанцу *PgConnectOptions* типа. Обе ове методе су садржане у *PgPoolOptions* и *PgPool* типу с тим да *PgPool* користи подразумијевану *pool* конфигурацију.

Инстанца *PgPool* типа се користи за сигурно извршавање упита од стране више нити. Она пружа једноставан интерфејс за слање упита а интерно користи ред конекција ка бази података. Користи примитиве конкурентности да синхронизује приступ овим конекцијама. Након омотавања инстанце *PgPool* типа у *AppState* и просљеђивања веб серверу, он може слободно да је клонира и шаље у више *request handler*-а јер она интерно користи *Arc<T>* тип који клонирање врши копирањем референце на *pool* конекција. Овиме се постиже кориштење само једног менаџера *pool*-а конекција од стране више *request handler*-а.

За креирање параметризованих упита користи се *query* функција која прима упит као стринг и враћа *Query* тип. Параметри у *Postgres* стрингу су представљени као $1, $2 итд. Везивање аргумената за параметре SQL упита се врши уланчавањем позива *bind* методе која прима тип који имплементира *sqlx::Encode* и *sqlx::Type*. Сви стандардни типови као што су *String, &str, i32* итд. имплементирају *sqlx::Type trait* тако што бивају мапирани у одговарајуће SQL типове (*TEXT, VARCHAR, INTEGER*). Такође, они имплементирају и *sqlx::Encode trait* што одређује како се ови Rust типови серијализују у одговарајући бинарни формат специфичне базе података. Ако колона у бази дозвољава *NULL* вриједност може се искористити аргумент *Option<T>* типа као параметар упита.

У овом пројекту је кориштен *query!* макро који као аргументе прима SQL упит и произвољан број аргумената упита. Он врши позив *query* функције и аутоматски додаје аргументе упиту *bind* методом. Затим враћа формирани упит у виду *Query* типа. Примјер једне методе из репозиторијума *User Identity* сервиса која користи *query!* макро дат је на сљедећој слици:

```
pub async fn get_user_by_email(&self, email: &str) -> sqlx::Result<UserEntity> {
    query!(
        r#"SELECT id,
           email,
           pwd_hash,
           user_type as "user_type: UserType",
           FROM wa_user WHERE email = $1"#,
        email
     ).fetch_one(&self.db_pool)
         .await
         .map(|row| UserEntity {
```

```
                id: row.id,
                email: row.email.map(UserEmail),
                pwd_hash: row.pwd_hash.map(|h| UserPassword(SecretString::from(h))),
                user_type: row.user_type,
            })
    }
```

Слика 5.12 Примјер репозиторијума који користи *query!* макро

Ова метода прима *email* стринг који се користи као аргумент параметризованог упита у *query!* макроу. Упит је дефинисан као *raw* стринг литерал кориштењем r#упит# формата да би се постигао аутоматски *escaping* недозвољених карактера и једноставан вишелинијски приказ.

За слање упита који врше писање у базу података користи се *execute* метода над *Query* инстанцом. За слање упита који само врше добављање резултата могу се користити *fetch, fetch_one, fetch_optional,* и *fetch_all* методе. Ове методе као аргумент примају референцу на *PgPool* инстанцу. Резултате ових метода је могуће мапирати на сљедеће начине:

1. Кориштењем *query* функције која враћа генерички *PgRow* тип. Податке овог типа је потребно мануелно екстрактовати кориштењем *try_get("column name")* позива методе. Овај начин не гарантује никакву сигурност током компајлирања тога да ли Rust тип може представити резултантне податке. Ово значи да ако дође промјене у бази (нпр. промјена назива колоне резултата) а не изврши се промјена аргумента *try_get* методе десиће се грешка током извршавања.

2. Кориштењем *query_as!* макроа чији је један аргумент одредишни тип у који ће резултат бити мапиран. Називи поља одредишног Rust типа морају одговарати именима колона резултата. Ако су сва поља стандардни типови мапирање се врши аутоматски јер стандардни типови имплементирају *sqlx::Type* и *sqlx::Decode trait*. Ако се користи типски-вођен развој (нпр. користи се *UserEmail(String)* тип као поље одредишног типа упита) потребно је да ово поље имплементира *sqlx::Type trait*. У ове сврхе може се искористити *derive* макро на начин да се испред поља наведе *#[derive(sqlx::Type)]*. Он током компајлирања аутоматски генерише *sqlx::Type* имплементацију за типове који као поља садрже стандардне типове (као што је *UserEmail(String)*). Такође, током компајлирања макро *query_as!* омогућава провјеру да ли наведени Rust тип може представити резултат упита. Он ово изводи тако што генерише код којим се клијент повезује са базом током компајлирања. Ако дође до неусклађене промјене назива поља у бази или коду десиће се грешка при компајлирању.

3. Кориштењем *query!* макроа који омогућава највећи ниво контроле јер ствара додатни слој у којем је могуће мапирати резултат у било који тип и на било који начин. Ово постиже тако што креира анонимну структуру

коју је онда могуће мапирати директним приступом њеним пољима. Ово мапирање се врши у *map* методи као у примјеру 5.12. Овај приступ је погодан када је потребно вршити додатне трансформације над пољима анонимне структуре прије него што буду мапиране у одредишни тип као што је то случај са *pwd_hash* пољем у примјеру. Као и *query_as!* макро врши провјеру током компајлирања да ли резултат може бити мапиран у одредишни тип с тим да имена колона не морају да одговарају именима поља одредишног типа јер постоји додатни слој. Поља анонимне структуре и даље морају да одговарају именима колона (у примјеру кориштење *row.id* израза значи да ће компајлирање бити успјешно само ако постоји колона резултата са називом *id*).

Додатно макрои *query!* и *query_as!* омогућавају и синтаксну и семантичку верификацију упита тако што могу аутоматски да се повежу са базом података током компајлирања. Да би се ова верификација могла извршити потребно је подесити *DATABASE_URL* варијаблу окружења и имати активну базу података са одговарајућом шемом. Да би се ово избјегло може се користити *sqlx* CLI алат који омогућава генерисање *.sqlx* кеша који садржи све неопходне податке за синтаксну и семантичку верификацију упита без потребе за постојањем активне базе података.

5.3.5. Клијент *RabbitMQ* брокера

За имплементацију клијента *RabbitMQ* брокера порука искориштена је популарна *lapin* библиотека. За креирање конекције искориштена је *connect_with_config* функција која као параметре прима URI конекциони стринг, клијентска подешавања везана за конекцију у виду *ConnectionProperties* типа и TLS подешавања у виду *TLSConfig* типа. У овом пројекту су кориштена подразумијевана подешавања везана за конекцију док се за подешавање TLS-а користе екстерни конфигурациони фајлови. У овим фајловима се спецификује да ли је омогућен TLS, mTLS и путање до фајлова потребних сертификата.

Након што је успостављена конекција и креирана *Connection* инстанца, могуће је путем *on_error* методе подесити *callback* функцију која се позива у случају грешака као што су: прекид конекције услијед мрежних проблема, затварање конекције од стране сервера, TLS грешака итд. Над *Connection* инстанцом, за креирање канала кориштена је *create_channel* метода која враћа *Channel* тип. Као и *Connection* тип, омогућава подешавање *callback* функције али се она у овом случају позива ако дође до грешке на нивоу канала као што су: декларисање реда или *exchange*-а са невалидним параметрима, конзумирање из непостојећег реда, недозвољен приступ реду или *exchange*-у итд.

За декларисање *exchange*-а кориштена је *exchange_declare()* метода над *Channel* инстанцом. Она као параметре прихвата назив *exchange*-а и његову врсту као што су *direct, fanout* и *topic*. У овом пројекту је кориштена *topic* врста да би се омогућило додавање нових сервиса без потребе за измјеном *publisher*-а. Сви могу да се претплате на *topic* и да добију поруку у виду догађаја (нпр. *UserRegisteredEvent*) и да у складу са

њим прилагоде своје перзистентно стање. Метода *exchange_declare* као параметре прихвата и кључ-вриједност парове у виду *FieldTable* структуре за додатна AMQP подешавања. Такође ту је и параметар типа *ExchangeDeclareOptions* за додатне опције као што су *durable* и *auto-delete*.

Извршавање акције *exchange_declare* методом је идемпотентно тако да је позивају и *UserIdentity* и *UserPreferences* сервис приликом покретања. *UserPreferences* сервис додатно приликом покретања, путем канала, декларише ред *queue_declare* методом а затим га везује за *exchange* методом *queue_bind*. Примјер везивања реда за *exchange* је приказан на сљедећој слици:

```
let res = channel
    .queue_bind(
        "user_preferences_service_queue",
        "user_events",
        "user.registered.#",
        QueueBindOptions::default(),
        FieldTable::default(),
    ).await;
}
```

Слика 5.13 Примјер везивања реда *queue_bind* методом

Први аргумент *queue_bind* методе је назив претходно декларисаног реда док је други аргумент назив *exchange*-а. Трећи аргумент је кључ рутирања *user.registered.#* који ће бити кориштен за одређивање које поруке послане у овај *exchange* ће бити смјештане у овај ред. У примјеру, то су све поруке чији кључ рутирања почиње са *"user.registered."* стрингом.

UserIdentity сервис као *publisher* догађаја објављује догађаје путем канала у претходно декларисани *exchange* методом *basic_publish* (слика 5.14).

```
let publisher_confirm = match channel
    .inner
    .basic_publish(
        "user_events",
        "user.registered.standard",
        BasicPublishOptions::default(),
        &*payload,
        BasicProperties::default()
            .with_delivery_mode(2)
            .with_content_type("application/json".into())
            .with_headers(headers),
        ).await {
            Ok(confirm) => confirm,
            Err(e) => {
                return Err(BrokerError::PublishingError("..."));
            }
        };
```

Слика 5.14 Примјер објаве догађаја *basic_publish* методом

Аргументи ове функције су назив одредишног *exchange*-а, кључ рутирања који је у овом случају *user.registered.standard.* Претходно наведени

user_preferences_service_queue ред у примјеру 5.13 ће да прими *payload* поруку која је у виду вектора бајтова. Креирање овог вектора бајтова постигнуто је помоћу *serde_json::to_vec* функције која серијализује структуру догађај (у овом примјеру *UserRegisteredEvent*) у бинарно енкодовани JSON стринг. Остали параметри *basic_publish* методе су *BasicPublishOptions* који служи за подешавање додатних опција на страни сервера, те *BasicProperties* за додавање метаподатака у саму поруку. У овом пројекту је додан *delivery_mode* метаподатак којим се подешава перзистирање поруке због тога што је могуће да дође до неконзистентног стања базе података *UserIdentity* и *UserPreferences* сервиса ако се деси рестарт брокера а послата порука није на вријеме достављена конзументу. Такође додан је и тип садржаја метаподатака као *application/json* те заглавља која су кориштена за додавање *tracing* података. Ови подаци служе за корелацију свих логова унутар једног пословног задатка који је иницирана корисничким захтјевом.

Као и све претходно наведене методе и функције *lapin* библиотеке и *basic_publish* је асинхроне природе тако да не долази до блокирања нити *async runtime*-а. Ова метода враћа *handler* (у примјеру *publisher_confirm*) који се може искористити за асинхроно примање обавјештења од стране брокера о успјешности објављивања поруке.

На страни конзумента се врши инстанцирање *Consumer* типа методом *basic_consume* над *Channel* инстанцом. *Consumer* тип имплементира *Stream trait* који представља широко кориштени асинхрони итератор у Rust-у. Позивање његове *next* методе враћа *future* над којим се користи *await* кључна ријеч да се асинхроно чека на сљедећу поруку од брокера (слика 5.15).

```
let mut consumer = channel
    .inner
    .basic_consume(
        "user_preferences_service_queue",
        "preferences_service_consumer_tag",
        BasicConsumeOptions::default(),
        FieldTable::default(),
    ).await
    .unwrap();

while let Some(delivery_result) = consumer.next().await {
    let Ok(delivery) = delivery_result else {
        continue;
    };
    match delivery.routing_key.as_str() {
        "user.registered.anonymous" => action1();
        "user.registered.standard" => action2();
    }
    delivery.ack(BasicAckOptions::default()).await
            .map_err(|e| tracing::error!("Failed to ack: {}", e)).ok();
}
```

Слика 5.15 Примјер конзумирања порука

Да би се постигло конзумирање и асинхроно чекање неограниченог броја порука, у примјеру се користи конструкција *while let* која извршава блок кода све док *stream* не врати грешку у виду *None* енума. Овиме је постигнута бесконачна петља све док не дође до критичне грешке везане за конзумента као што је прекид конекције. Сваки пут када је конзумирање поруке успјешно *while let* ће извршити додјељивање резултата у *delivery_result* инстанцу. Тип ове инстанце је *DeliveryResult* који енкапсулира резултат који може бити успјешан или фаличан са грешком везаном за специфичну поруку. У случају грешке наставља се конзумирање нових порука. Ако је резултат успјешан из резултат ће бити екстрактован *delivery* инстанца. Она је типа *Delivery* који представља примљену поруку и пружа методу за добављање кључа рутирања. На основу кључа рутирања извршава се одговарајућа акција уз помоћ *match* конструкције. Прије самог кориштења садржаја поруке који је у овом случају бинарно енкодовани JSON стринг, потребно је извршити његову десеријализацију у Rust структуру функцијом *serde_json::from_slice. Delivery* тип пружа методе за слање потврда брокеру о успјешности процесирања поруке као што су *ack(), nack()* и *reject()*.

5.3.6. Аутентификација и остали сигурносни аспекти

За рад са JSON веб токенима искориштена је популарна *jsonwebtoken* библиотека. За креирање JWT-а искориштена је *encode* функција која прихвата *Header* инстанцу, тврдње у виду *custom* типа те кључ за потписивање у виду *EncodingKey* типа. *Header* тип служи за додавање података заглавља у виду кључ-вриједност парова. У овом пројекту он је искориштен за додавање информације који алгоритам је искориштен приликом потписивања података из токена. Тип који садржи тврдње мора да имплементира *serde::Serialize trait* да би могао бити уграђен у токен. Имена поља овог типа са својим вриједностима аутоматски постају тврдње у токену. Вриједности кључа могу бити симетрични и асиметрични кључеви (RSA, HMAC, EdDSA итд.) учитани из *.der* или *.pem* формата.

За декодовање и валидацију токена искориштена је *decode* генеричка функција. Њен генерички параметар је одредишни тип који садржи тврдње. Он мора да имплементира *serde::Deserialize trait*. *Encode* функција као параметре прихвата токен у виду стринга, кључ за валидацију те *Validation* инстанцу. Ова инстанца омогућава подешавање разних валидација које се извршавају након декодовања токена као што су: искориштени алгоритам за потписивање, постојање поља са одређеним вриједностима као што су издавалац и субјекат итд. Аутоматски врши провјеру да ли је токен истекао користећи *exp* поље. Примјер функције која валидира токен у *UserPreference* сервису дат је на сљедећој слици:

```
pub fn validate_token(&self, token: &str) -> Result<Claims, Error> {
    let mut validation = Validation::new(Algorithm::EdDSA);
    validation.set_issuer(&[self.jwt_settings.issuer_name.as_str()]);

    let token_data = decode::<Claims>(
        token,
        &self.public_key,
        &validation
```

```
        )?;

        Ok(token_data.claims)
    }
```

Слика 5.16 Функција за валидацију JWT-а

Да би се спријечиле неке врсте напада експлицитно је наведен алгоритам тако да ће валидација бити успјешна само ако је токен потписан EdDSA алгоритмом. Додатно, *Validation* инстанца је подешена са конкретним издаваоцем који је у овом случају *UserIdentity* сервис. *Decode* функцији се просљеђује токен, јавни EdDSA кључ те *Validation* инстанца. Повратна вриједност су екстрактоване тврдње из токена.

Сам јавни кључ се добавља са јавно доступног *endpoint*-а у виду *jwks* фајла. На овај начин је успостављена инфраструктура за аутентификацију корисника. *UserIdentity* сервис креира и потписује токене својим приватним кључем те објављује свој јавни кључ на јавно доступном *endpoint*-у. Други сервиси (у овом случају *UserPreferences* сервис) користе овај јавни кључ и презентовани JWT приликом захтјева да аутентификују корисника.

За хеширање лозинки употребом модерног Argon2 алгоритма искориштена је *argon2* библиотека. Пружа структуру *Argon2* која представља Argon2 контекст. Она садржи методу *hash_password* која прихвата лозинку као низ бајтова те *SaltString* који се може генерисати помоћном *password_hash* библиотеком (слика 5.17). Ова функција враћа стринг у модуларном криптографском формату који садржи све потребне информације као што су *salt* и верзија алгоритма за каснију верификацију лозинке.

```
    pub fn hash_password(password: &str) -> Result<String, Error> {
        let salt = SaltString::generate(&mut OsRng);
        let argon2 = Argon2::default();

        Ok(argon2.hash_password(password.as_bytes(), &salt)?.to_string())
    }
```

Слика 5.17 Функција за хеширање лозинке

Прије верификације лозинке потребно је парсирати сачувану хеш вриједност лозинке из Argon2 формата употребом *PasswordHash* структуре (слика 5.18). Затим је потребно искористи *verify_password* методу *Argon2* инстанце која прихвата лозинку која се верификује у виду низа бајтова и *PasswordHash* инстанцу.

```
    pub fn verify_password(pwd: &str, stored_pwd_hash: &str) -> Result<bool, Error> {
        let parsed_pwd_hash = PasswordHash::new(stored_pwd_hash)?;

        Ok(Argon2::default().verify_password(pwd.as_bytes(), &parsed_pwd_hash).is_ok())
    }
```

Слика 5.18 Функција за верификацију лозинке

5.3.7. Опсервабилност

За генерисање телеметријских података искориштен је *tracing* развојни оквир. Генерално, телеметријски подаци дају информације о покренутој апликацији и бивају аутоматски прикупљани. На овај начин апликација постаје опсервабилна. Ове информације се касније могу анализирати да би се утврдило у каквом је стању апликација била у неком тренутку времена.

Основна структура *tracing* библиотеке је *Span* која представља период извршавања дијела програма. Могуће је ући у *span* позивањем његове *enter* методе након чега је могуће биљежити информације у виду догађаја. Ове информације су у формату кључ-вриједност парова и оне бивају аутоматски повезане са тренутно активним *span*-ом. За генерисање догађаја се користе макрои као што су *tracing::info!* и *tracing::error!*. *Span*-ови могу бити угњеждени и формирати структуру стабла. Могуће је ући и изаћи из *span*-а произвољан број пута. С друге стране затварање *span*-а је трајно и врши се аутоматски након што повратна вриједност *enter* методе изађе из опсега. Да би се спријечио мануелни улазак у *span* могуће је декорисати функцију или методу са *instrument* макроом што омогућава аутоматски улазак у *span* сваки пут када је она позвана. Ако се ради о асинхроној функцији улазак у *span* ће се десити сваки пут када *async runtime* буде извршавао ову функцију, док ће се излазак десити сваки пут када он паузира њено извршавање. На овај начин се постиже једноставно кориштење *tracing* библиотеке у асинхроном окружењу као што је веб сервер. Примјер инструментализоване методе и генерисања телеметријских података дат је на сљедећој слици:

```
#[tracing::instrument(
    name = "Auth service - get user info function",
    skip(self, access_token)
)]
pub async fn get_user_info(
    &self,
    access_token: String
) -> Result<UserInfoResponse, UserIdentityServiceError> {
    let user_id = match self.jwt_service.validate_token(access_token.as_str()) {
        Ok(claims) => {
            tracing::info!("Access token is valid. Extracting user id.");
            claims.sub
        },
        Err(error) => {
            tracing::error!("Failed to validate token: {:?}", error.to_string());
            return Err(ServiceError::Unauthorized("Invalid token".to_string()));
        }
    };
/* ... */
}
```

Слика 5.19 Примјер инструментализоване методе

Instrument макро као аргументе прихвата назив *span*-а и *skip* директиву која омогућава изостављање аутоматског додавања аргумената функције као података *span*-а. У функцији *get_user_info()* изостављена је *AuthService* инстанца над којом је позвана

инструментализована метода јер садржи поља која нису од телеметријског значаја те *access_token* из сигурносних разлога. Резултат позивања *validate_token* методе над *jwt_service* инстанцом може бити успјешан или неуспјешан и у складу с тим, врши се креирање догађаја уз помоћ *tracing::info!* и *tracing::error!* макроа.

Tracing библиотека генерише *span*-ове а за њихово прикупљање потребно је користити *tracing-subscriber* библиотеку. Она пружа *Registry* тип који се глобално претплати на све генерисане *span*-ове те их прикупља на централно мјесто у коду. Додатно, овај претплатник може да прикупља све *span*-ове, догађаје и логове других кориштених библиотека у пројекту. Ове библиотеке морају да користе *tracing* библиотеку као API за генерисање *span*-ова. Ако оне умјесто *tracing*-а користе *log* стандардну библиотеку за генерисање логова потребно је регистровати *Registry* као претплатника на све генерисане логове употребом *LogTracer* типа из *tracing_log* библиотеке. *Registry* користи *Layer* тип што омогућава процесирање *span*-ова у више слојева. Као *Layer*, у овом пројекту је кориштен слој из *tracing_bunyan_formatter* библиотеке који форматира све *span*-ове и логове у JSON формат и шаље их на стандардни излаз. Додатно, кориштен је и *OpenTelemetryLayer* из *tracing_opentelemetry* библиотеке који трансформише све *tracing* податке у *OpenTelemetry* формат и периодично их експортује на *tracing backend*.

Да би се могао пратити цијели логички ток захтјева унутар једног сервиса, потребно је креирати коријенски *span* са *request_id* пољем. Унутар овог *span*-а ће бити креирани сви *subspan*-ови током позива инструментализованих метода и функција и сваки до њих се садржати *request_id* поље. Ово је аутоматски изведено додавањем *TracingLogger* HTTP сервер *middleware*-а из *tracing_actix_web* библиотеке.

Због тога што се ради о дистрибуираном систему гдје крајњи кориснички захтјев може да обухвати позиве више микросервиса потребно је извршити корелацију *span*-ова сваког од ових сервиса. Ово се изводи такозваном *Context Propagation* техником. Први сервис у ланцу сервиса креира коријенски *span* а затим приликом слања захтјева другим сервисима додаје одређена HTTP заглавља са информацијама које су потребне серверу да изврши корелацију са својим *span*-овима. Да би се ово постигло аутоматски, кориштен је *TracingMiddleware* над HTTP клијентом из библиотеке *reqwest_tracing*. Такође, потребно је подесити глобално доступни *trace context propagator* којег овај *middleware* користи за креирање HTTP заглавља. На серверској страни *TracingLogger middleware* аутоматски врши екстракцију *tracing* података из HTTP заглавља те њихово везивање за коријенски *span* захтјева. Уз помоћ ових података *tracing backend* може адекватно да прикаже цијели ток захтјева који може имати комплексну структуру стабла.

Конкретни кориштени *tracing backend* у овом пројекту је *Jaeger* систем. Уз сваки микросервис, унутар исте машине или *Pod*-а, извршава се и *OpenTelemetry* агент. Он растерећује апликациони контејнер свих операција везаних за експорт *tracing* података на удаљени систем. Ови агенти експортују податке на централни *OpenTelemetry Jaeger* колектор који има улогу бафера података. Он ове податке обрађује, индексира и шаље у *ElasticSearch* складиште података. Ово складиште се

користи од стране *Jaeger UI* контејнера који приказује *tracing* податке путем богате веб апликације. Приступ овој веб апликацији је заштићен *OAuth 2.0* прокси сервером који дозвољава приступ само ако захтјев садржи исправан JWT унутар сесијског колачића. Овај колачић се креира након што се корисник аутентификује на *Keycloak Identity Provider* систем.

5.3.8. Конфигурабилност

За конфигурисање апликације искориштена је популарна *config* библиотека. Она омогућава хијерархијско организовање конфигурације како из различитих екстерних извора тако и из самог кода. Неки од подржаних екстерних извора су варијабле окружења и фајлови стандардних формата као што су TOML, JSON, YAML итд.

За креирање структуре *Config* која представља конфигурацију користи се *ConfigBuilder* тип. Уланчавањем позива методе *add_source()* он омогућава хијерархијско додавање више извора конфигурационих података гдје извори додани касније имају већи приоритет. Сви ови подаци се спајају тако да је могућа ситуација да дође до њиховог преписивања и допуњавања. Након додавања свих извора потребно је позвати *build()* методу која иницира све евентуалне I/O операције. Ако је резултат успјешан потребно је десеријализовати конфигурационе податке у тип који садржи све конфигурационе податке као поља и који имплементира *serde::Deserialize trait*.

У овом пројекту је кориштен базични YAML фајл за основне конфигурационе податке као што су назив базе података или назив *tracer*-а а затим су додани специфични подаци окружења за извршавање (*dev, prod*) такође из YAML фајла који надопуњују и преписују основне податке. Одређивање о којем окружењу се ради се врши на основу вриједност APP_ENV варијабле окружења (слика 5.20).

```
let environment: RunningEnvironment = std::env::var("APP_ENV")
        .unwrap_or_else(|_| "prod".into())
        .try_into()
        .expect("Failed to parse env. variable with name APP_ENV.");

let env_filename = format!("configuration_{}.yaml", environment.as_str());

let settings = Config::builder()
    .add_source(File::from(
        configuration_directory.join("configuration_base.yaml"),
    ))
    .add_source(File::from(
        configuration_directory.join(env_filename),
    ))
    .build()?;

let settings = settings.try_deserialize::<Settings>();
```

Слика 5.20 Примјер учитавања конфигурације

Прво се инстанцира енум *RunningEnvironment* на основу APP_ENV варијабле окружења кориштењем *var()* функције из стандардне библиотеке. Ако ова варијабла није подешена подразумијевано се подешава продукционо окружење. Затим се на

основу *RunningEnvironment* инстанце формира назив конфигурационог фајла. Овај назив се користи за креирање *File* инстанце користећи *File::from()* функције. Затим се овај фајл додаје као извор конфигурације уз помоћ *add_source()* методе након што је на овај исти начин учитана и основна конфигурација. Након позива *build()* методе креира се *Config* инстанца коју је потребно десеријализовати генеричком функцијом *try_deserialize<T>()*.

5.4. Развијено рјешење

Чињенице које су помогле брзи развој микросервиса у програмском језику Rust су сљедеће:

1. Кориштење практичног веб развојног оквира са функционалностима као што су рутирање, *dependency injection*, *middleware*-и.
2. Кориштење широко распрострањеног развојног оквира за серијализацију и десеријализацију са макроима за једноставно, брзо премошћавање разлика између Rust типова и разних формата података.
3. Кориштење дескриптивних компајлерских порука приликом API-вођеног дизајна.

Додатно, постигнуте се сљедеће пожељне особине сваког сервиса:

1. Високе перформансе уз помоћ библиотека са асинхроним интерфејсом и самом чињеницом да Rust не користи *garbage collector.*
2. Стабилност током извршавања захваљујући строгом систему типова, управљања грешкама и потребе за експлицитном валидацијом варијабли које не морају да имају вриједност.
3. Тривијално скалирање јер сервиси не чувају никакво стање клијента у својој меморији те приликом комуникације са екстерним компонентама користе *connection pooling*.
4. Конфигурабилност тако да је апликацију могуће једноставно прилагодити различитим *deployment* окружењима.
5. Олакшан проналазак грешака јер је цијели систем опсервабилан путем богатог веб корисничког интерфејса.

Због тога што је *Weather Aggregator* систем реализован микросервисном архитектуром постигнуте су сљедеће пожељне особине овог система:

1. Проширивост система новим сервисима као што су нови адаптер сервиси који користе друге провајдере временских података.
2. Поновна употребљивост сервиса у друге сврхе. Нпр. *UserIdentity* сервис је агностичан по питању тога у којој конкретној апликацији је кориштен

тако да се може користити у другим апликацијама које немају везе са апликацијом за агрегацију времена.

3. Независни развој по тимовима.
4. Независно скалирање. Нпр. агрегатор сервис може имати много веће оптерећење од *UserIdentity* сервиса.

6. Закључак

Микросервисна архитектура настаје као одговор на недостатке монолитне. Са својим модуларним дизајном, она постиже независан развој, испоруку, еволуцију, скалирање и избор технологије по цијену веће комплексности и дистрибуираности. Конкретни проблеми који се јављају су комплекснија комуникација, непоузданост мреже, дистрибуиране трансакције, лоша опсервабилност. Због тога микросервисни систем захтјева кориштење *lightweight* комуникације, *eventual consistency* приступа те постизање веће отпорности и детаљне опсервабилности. Такође, појављује се и проблем оперативне комплексности која се може ублажити контејнеризацијом и *DevOps* праксом.

Да би се ефикасно развила микросервисна апликација потребно је имати на располагању програмски језик са одговарајућим библиотекама и развојним оквирима. Један такав језик је Rust. Иако се ради о релативно новом програмском језику, он је довољно експресиван и садржи богат екосистем библиотека и развојних оквира за развој веб апликација. Са својим строгим системом типова, управљањем грешкама и експлицитном валидацијом варијабили које не морају да имају вриједност омогућава писање сигурних и поузданих апликација. Са својим *ownership* системом и непостојањем *garbage collector*-а омогућава писање апликација са високим перформансама. Његова мана је спор процес компајлирања. Међутим, употребом *cargo check* алата многе споре фазе компајлирања се не извршавају тако да је негативан ефекат на развој микросервиса минималан.

У практичном дијелу рада реализована је микросервисна веб апликација која илуструје кориштење програмског језика Rust. Кориштен је моћан веб развојни оквир са функционалностима као што су рутирање захтјева, *middleware*-и, *dependency injection*, управљање грешкама итд. Једноставно је извршено управљање и манипулација улазним и излазним подацима у разним форматима јер је кориштен генерички развојни оквир за серијализацију и десеријализацију. Комуникација са екстерним системима се изводи асинхроно користећи *connection pooling* тако да је одржана скалабилност микросервиса. Апликација је потпуно опсервабилна захваљујући богатом *tracing* екосистему. Такође, она је и конфигурабилна тако да се лако може прилагодити различитим *deployment* окружењима.

Крајњи резултат је ефикасна, сигурна, поуздана, скалабилна, поновно употребљива, потпуно опсервабилна и конфигурабилна микросервисна апликација. Можемо закључити да се уз помоћ Rust програмског језика може ефикасно развити микросервисна апликација с тим да је потребно имати већу дисциплину због *ownership* система и користити адекватне алате за скраћивање времена компајлирања.

Литература

[1] Mihajlo Savić, Internet Informacioni Sistemi. Univerzitet u Banjoj Luci, Elektrotehnički fakultet, 2024.

[2] M. Fowler, Patterns of Enterprise Aplication Architecture. Addison-Wesley, 2002.

[3] Thomas Erl, Service-Oriented Architecture: Analysis and Design for Services and Microservices. Prentice Hill 2017.

[4] Thomas Erl, SOA Principles of Service Design. Prentice Hall 2007.

[5] Oracle, Simple Object Access Protocol Overview, https://docs.oracle.com/cd/A97335_02/integrate.102/a90297/overview.htm, posjećeno 17.12.2024.

[6] UC Irvine, REST, https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm, posjećeno 20.12.2024.

[7] InfoQ, Hypermedia in RESTful applications, https://www.infoq.com/articles/mark-baker-hypermedia, posjećeno 26.1.2025.

[8] DZone, REST vs Messaging for Microservices, https://dzone.com/articles/rest-vs-messaging-for-microservices, posjećeno 3.1.2025.

[9] Amazon, REST vs GraphQL, https://aws.amazon.com/compare/the-difference-between-graphql-and-rest/, posjećeno 13.5.2025.

[10] Microsoft Corporation, .NET Microservices Architecture for Containerized NET Applications

[11] Databricks, ACID Transactions, https://www.databricks.com/glossary/acid-transactions, posjećeno 9.1.2025

[12] Amazon, What’s the Difference Between Docker Images and Containers,

https://aws.amazon.com/compare/the-difference-between-docker-images-and-containers, posjećeno 28.1.2025.

[13] Amazon, What is Containerization, https://aws.amazon.com/what-is/containerization/, posjećeno 19.5.2025.

[14] Amazon, What is DevOps, https://aws.amazon.com/devops/what-is-devops/, posjećeno 21.5.2025.

[15] Microsoft Corporation, Containerized Docker Application Lifecycle with Microsoft Platform and Tools

[16] Amazon, What is IaC, https://aws.amazon.com/what-is/iac/, posjećeno 21.5.2025.

[17] Amazon, What is Service Mesh, https://aws.amazon.com/what-is/service-mesh/, posjećeno 14.2.2025.

[18] Martin Fowler, Microservices, https://martinfowler.com/articles/microservices.html, posjećeno 7.5.2025.

[19] Particular.net, Fallacies of Distributed Computing, https://particular.net/blog/the-network-is-reliable, posjećeno 28.2.2025.

[20] Amazon, SOA vs Microservices, https://aws.amazon.com/compare/the-difference-between-soa-microservices/, posjećeno 3.3.2025.

[21] The Rust Programming Language, https://doc.rust-lang.org/book/, posjećeno 21.4.2025.